



National University of Computer and Emerging Sciences, Lahore



AGAHI: Adaptive Generation for Academic Hub and Information (An AI-Powered Study Companion for Pakistani University Students)

Zain Allaudin 23L-3036 BS(SE)

Zahra Amjad 23L-3072 BS(SE)

Syed Hadi Ali Akbar Zaidi 23L-3023 BS(SE)

Supervisor: Ms. Sarah Abid Khan

Final Year Project

October 10, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: AGAHI: Adaptive Generation for Academic Hub and Information
(An AI-Powered Study Companion for Pakistani University Students)

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: October 10,2026

Name: Zain Allaudin

Zain

Signature: .....

Name: Zahra Amjad

Zahra

Signature: .....

Name: Syed Hadi Ali Akbar Zaidi

Hadi

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

University students often struggle to manage different academic materials during exams. The lecture notes, textbooks, class notes, projects, and past papers are scattered on multiple media, so it might be hard to find suitable material, work on issues and effectively plan revisions. Current study apps are often useful for particular tasks, while general AI assistants may answer without relevant academic materials as a reference point. AGAHI (Adaptive Generation for Academic Hub and Information) is proposed as a web-based AI-powered study companion that provides a course-aware learning environment for university students. The system will process text-based PDF resources, including lecture slides, notes, textbooks, and past examination papers, and use Retrieval Augmented Generation (RAG) to produce responses grounded in the uploaded academic material. Where possible, generated responses will include source and page references to improve traceability. In addition to document-based question answering, AGAHI will provide automated summaries, flashcards, summary sheets, interactive quizzes, past-paper topic analysis, and a centralized learning dashboard. Quiz performance will be analyzed at the topic and sub-topic level to identify areas of weakness. The adaptive revision mechanism will use an approximate 70/30 distribution, allocating greater practice toward weaker topics while retaining previously learned material. A weakness heatmap, progress tracking, streaks, immediate feedback, and Pomodoro focus timers will further support the student's study process. The project is primarily targeted at undergraduate and postgraduate students at FAST-NUCES and similar universities in Pakistan. The initial system will support English-language, text-based PDF resources and focus on academic coaching, practice, revision, and progress tracking. AGAHI aims to provide a unified and course-aware study environment that helps students move from collecting academic material to understanding, practicing, and revising it in a more organized manner.

Executive Summary

Students in universities may have access to an extensive amount of academic material, such as lecture slides, textbooks, notes taken by themselves, assignments, and previous exams. These documents have all the information needed for preparing for exams but can be difficult to examine and study properly. In this case, students must identify relevant materials, select the topics that need more revision, go back to the previously studied materials, and answer questions while employing different tools for each task. The problem is made even more difficult by the fact that not all students comprehend the material equally, and one topic can be much harder and require more practice for one student than for the other AGAHI (Adaptive Generation for Academic Hub and Information) is proposed as a web-based AI-powered study companion for university students, particularly at FAST-NUCES and similar universities in Pakistan. The main purpose of AGAHI is to bring common study activities into one course-aware environment. Instead of functioning as a general chatbot, the system will work with the academic material uploaded by the student so that generated content remains relevant to the course material being studied. The system will accept text-based PDF resources such as lecture slides, notes, textbooks, and past examination papers. The uploaded documents will be processed and divided into meaningful content, topics, and subtopics. Retrieval Augmented Generation (RAG) will then be used to retrieve relevant information when a student asks a question. The retrieved content will be provided to the language model so that responses are based on the uploaded academic material. Where possible, responses will include the source document and relevant page number for verification. The 70/30 distribution is intended as a guideline for balancing improvement with retention rather than as a fixed rule for every student. Past examination papers will also be analyzed to identify recurring questions, topics, and patterns. This information will be combined with the student's performance data to provide a clearer view of both historically important topics and current weaknesses. Past examination papers will also be reviewed to find commonly repeated questions, important topics, and patterns. This information will then be considered along with the student's performance to identify topics that have been important in previous exams as well as areas where the student currently needs improvement.A centralized dashboard will allow students to track their progress through features such as a weakness heatmap, progress indicators, streaks, immediate quiz feedback, and customizable Pomodoro timers. The initial system will support English-language interfaces and text-based PDF documents, with a limit of 30 MB or 150 pages per file, whichever is reached first. Scanned or image-only PDFs, handwritten notes, and non-PDF files will be outside the document-processing scope. AGAHI will generate study content only from the material uploaded by the student and will not use external web searches for academic answers or generated study material. Integration with learning management systems, enrollment, registration, payment processing,

and other administrative functions is also outside the project scope. Initial work has included reviewing tools such as Quizlet, ChatPDF, and Notion AI, studying Retrieval Augmented Generation, finalizing the technology stack, setting up the Supabase project and initial database schema, and developing an initial PDF ingestion pipeline. Sample lecture slides and past papers have also been collected for testing. Overall, AGAHI aims to provide a single platform where students can organize their academic resources, understand course material, create study aids, practice questions, identify weak areas, and plan their revision in a more organized way. The system is intended to support students in making better use of their existing academic material rather than replacing instructors or university resources

Table of Contents

List of Figures **ix**

List of Tables **x**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.3.1 Definitions 2

1.3.2 Acronyms and Abbreviations 2

1.4 Conclusion 3

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goal (SDG) 7

2.7 Constraints 8

2.8 Business Opportunity 9

2.9 Stakeholders Description/ User Characteristics 10

2.9.1 Stakeholders Summary 11

2.9.2 Key High-Level Goals and Problems of Stakeholders 11

3 Literature Review / Related Work **14**

3.1 Definitions, Acronyms, and Abbreviations 14

3.2 Detailed Literature Review 16

3.2.1 Quizlet: Flashcard-Based Study Platform 17

3.2.2 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020) . . . 18
3.2.3 ChatPDF: Conversational Question Answering over a Single Document . . . 21
3.2.4 Turbo AI (formerly Turbolearn AI): An Integrated Note, Flashcard and Quiz Generator . . . 22
3.2.5 A Systematic Review of Automatic Question Generation for Educational Purposes 24
3.3 Literature Review Summary Table . . . 26
3.4 Conclusion . . . 30

4 Software Requirement Specifications 32
4.1 List of Features . . . 32
4.2 Functional Requirements . . . 32
4.3 Quality Attributes . . . 35
4.3.1 Reliability . . . 36
4.3.2 Maintainability . . . 36
4.3.3 Usability . . . 36
4.3.4 Correctness . . . 36
4.4 Non-Functional Requirements . . . 37
4.4.1 Availability . . . 37
4.4.2 Reusability and Extensibility . . . 37
4.4.3 Robustness and Fault Tolerance . . . 37
4.4.4 Security and Privacy . . . 37
4.4.5 Performance and Scalability . . . 38
4.5 Assumptions . . . 38
4.6 Use Cases . . . 38
4.7 Hardware and Software Requirements . . . 45
4.7.1 Hardware Requirements . . . 45
4.7.2 Software Requirements . . . 45
4.8 Graphical User Interface . . . 45
4.9 Database Design . . . 52
4.9.1 ER Diagram . . . 52
4.9.2 Data Dictionary . . . 52
4.10 Risk Analysis . . . 54
4.10.1 Technical Risks . . . 54
4.10.2 Risks Associated with Data and Privacy . . . 54

4.10.3 Risks of Development and Project 54

5 High-Level and Low-Level Design 55

5.1 System Overview 55

5.1.1 Document Ingestion and Extraction of Topics 55

5.1.2 Grounded Question Answering 56

5.1.3 Study Aid Generation 56

5.1.4 Adaptive Practice and Mastery Tracking 56

5.1.5 Past-Paper Analysis and Revision Guidance 56

5.1.6 Focus and Engagement Features 57

5.1.7 Basic Design Approach 57

5.2 Design Considerations 58

5.2.1 Assumptions and Dependencies 58

5.2.2 General Constraints 59

5.2.3 Goals and Guidelines 62

5.2.4 Development Methods 63

5.3 System Architecture 65

5.3.1 Subsystem Architecture 67

5.4 Architectural Strategies 68

5.4.1 Programming Language and Framework Selection 68

5.4.2 Database Strategy 68

5.4.3 AI Pipeline Architecture 68

5.4.4 Concurrency Strategy 68

5.4.5 User Interface Paradigm 69

5.4.6 Version Control 69

5.5 Domain Model/Class Diagram 70

5.6 Sequence Diagrams 71

5.7 Policies and Tactics 75

5.7.1 Coding Environment 75

5.7.2 Coding Guidelines and Conventions 76

5.7.3 Error Detection and Recovery 76

5.7.4 Testing Strategy 76

5.7.5 Extensibility 76

5.7.6 Software Maintenance 76

List of Figures

2.1 Quality Education 7

4.1 Signup Page 46
4.2 Login Page 47
4.3 Library Page 47
4.4 Ask AGAHI Page 48
4.5 Summary Page 48
4.6 Quiz Page 49
4.7 Exam Insights Page 49
4.8 Progress Page 50
4.9 Dashboard Page 50
4.10 Focus Timer Page 51
4.11 ER Diagram 52

5.1 Architecture Diagram 65
5.2 Application Tier Subsystem 67
5.3 Class Diagram 70
5.4 System Level Sequence Diagram 71
5.5 Generate Study Aids 72
5.6 Attempt Practice Quiz 73
5.7 Analyze Past Examination Paper 74
5.8 Use Pomodoro Study Timer 75

List of Tables

2.1 Summary of AGAHI stakeholders, their roles, and how they are involved with the system. 12
2.2 High-level goals of each stakeholder and the problems they currently face that AGAHI aims to address. 13

3.1 Summary of existing study tools and products reviewed 27
3.2 Summary of research on retrieval-augmented generation and grounded answering . 28
3.3 Summary of research on automatic question generation and learning science 29

4.1 Use Case (UC-01) – Sign Up 39
4.2 Use Case(UC-02) – Login 39
4.3 Use Case (UC-03) – Upload Academic Resource 40
4.4 Use Case (UC-04) – Ask Academic Question 41
4.5 Use Case (UC-05) – Generate Study Aids 41
4.6 Use Case (UC-06) – Attempt Practice Quiz 42
4.7 Use Case (UC-07) – Analyze Past Examination Paper 43
4.8 Use Case(UC-08) – View Study Progress 43
4.9 Use Case (UC-09) – Receive Revision Recommendations 44
4.10 Use Case (UC-10)– Use Pomodoro Study Timer 44
4.11 Data Dictionary Table 53

Chapter 1 Introduction

An ongoing flow of information is an inherent feature of university studies. Learning in university is not limited to attending classes; students need to review the subject matter, establish correlations between ideas from various sources, prepare for evaluations, and revisit what they have learned in lectures before their exams. Since there is more and more information to study throughout the semester, students need to develop a solid strategy for managing their learning process and revision. The evolution of study habits has happened along with the development of electronic resources. Academic information is now stored and scanned using electronic files, and various online applications are used for such tasks as taking notes, answering questions, or reviewing topics. While these electronic resources offer a certain level of usefulness, students need to combine information from different applications and use each tool according to their learning needs. In this regard AGAHI (Adaptive Generation for Academic Hub and Information) is introduced as a navigation tool for improving academic learning process. This idea is based on the concept of students taking advantage of the sources they already possess and getting assistance at certain points of their studying process.

1.1 Purpose of this Document

This document aims to define AGAHI (Adaptive Generation for Academic Hub and Information), an AI-based educational tool that can help students in organizing and preparing their study materials. The document describes the problem that the project is addressing, its aim and objectives, scope, requirements, the chosen approach, design, development, testing, and results. It also provides a standardized basis to understand how AGAHI is supposed to work, how various components of the system impact the educational process, as well as the limitations and assumptions of the system so that the purposes of application of the system and its limitations are clear.

1.2 Intended Audience

AGAHI is primarily intended for undergraduate and postgraduate university students who need to manage and revise a large amount of academic material during their courses. The system is particularly aimed at students who use lecture slides, notes, textbooks, and past examination papers as part of their regular study and examination preparation. The platform will be designed to support students with different levels of understanding of a subject. Students can use their own course material to obtain relevant explanations, generate study resources, practice through quizzes, identify weaker topics, and organize their revision. AGAHI is especially useful for students who find it difficult to determine which topics

require more attention or who currently rely on multiple separate tools for different study activities. The initial target users are students at FAST-NUCES Lahore and, more broadly, students enrolled in universities in Pakistan.

1.3 Definitions, Acronyms, and Abbreviations

1.3.1 Definitions

AGAHI: The name of the proposed study companion that helps students with their academic material.

Artificial Intelligence: A branch of computer science which enables machines to execute a task which usually requires human intelligence.

Retrieval Augmented Generation: A method involving finding the necessary information from the provided data in order to produce an answer.

Large Language Model: An AI model that can understand questions and generate responses in natural language.

User Interface: The part of the system through which students use and interact with AGAHI.

Pomodoro Timer: A study technique in which study time is divided into focused work periods and short breaks.

Weakness Heatmap: A visual representation that shows the topics and sub-topics in which a student needs more practice.

Adaptive Revision: This method implies adjusting the amount of practice based on the productivity of a student.

1.3.2 Acronyms and Abbreviations

AI: Artificial Intelligence

RAG: Retrieval Augmented Generation

PDF: Portable Document Format

LLM: Large Language Model

UI: User Inteface

FAST NUCES: National University of Computer and Emerging Sciences

1.4 Conclusion

In conclusion, this chapter explained the background of the project and the need for proper organization with respect to studying and the revision of academic work. The concept of AGAHI was explained along with the purpose and audience of the document. Furthermore, the important terms, definitions, acronyms, and abbreviations used in the report were also stated. This chapter thus provided the basis needed to understand the project, while the next chapters will contain details about its vision, scope, requirements and other aspects in greater detail.

Chapter 2 Project Vision

This chapter presents the vision behind AGAHI by introducing its problem domain, the problem it aims to address, and the main challenges related to it. It also defines the goals, objectives, and scope of the proposed system.

2.1 Problem Domain Overview

University students have to deal with a lot of study information during their academic tenure. This information is present in lectures, books, teachers' notes, assignment papers, and previous exams. Many students are required to put in a lot of effort in organizing, filtering, and using this information. The difficulty is not limited to finding information. Students also need to determine which topics they understand well, which topics require more practice, and which areas have historically been important in examinations. Conventional study plans generally follow the same sequence for all students, even though individual students may have different levels of understanding. Recent AI-based educational tools have made activities such as summarization, question answering, and quiz generation easier. However, general-purpose AI assistants may not be grounded in the specific material of a student's course. As a result, their responses may contain information that is not directly relevant to the uploaded course resources or may use terminology different from that used in the course. AGAHI addresses this problem domain by combining document-based AI assistance with personalized learning support. The system is designed to work with the student's own academic resources, generate study material from those resources, analyze quiz performance, identify weak areas, and support targeted revision within a single platform.

2.2 Problem Statement

University students often have plenty of course material available to them, but managing this material and using it effectively for preparation is difficult. Information is usually spread across different documents, while students have to decide for themselves what is important, what they already understand, and what needs more revision. Existing AI tools can assist with questions and summaries, but their responses are not necessarily limited to the student's own course material. There is therefore a need for a study system that can work with a student's academic resources while also considering the student's learning progress and areas of difficulty.

2.3 Problem Elaboration

University students usually have to study from various resources, like lecture slides, notes, textbooks, tasks, and old exams. Searching through the materials and finding the required information can be difficult and require quite a bit of time, especially during exam preparations. Often students have to read many documents just to study one subject. Moreover, generative AI tools do not always give answers that match the student's study material. The answers may include information that wasn't provided in the course or use other terminology. As a result, students have to check the results with their course notes before using AI for exam preparations. Preparing the study material takes up a lot of time as well. Students often have to create their own summaries, flashcards, and exercises before being able to study. Additionally, when there are too many courses to prepare for, the amount of time left for studying is reduced. The process of preparing revision materials is yet another lengthy process. Students often have to create their own summaries, flashcards, and practice questions ahead of time before they can actually begin the process of revision. It becomes harder to dedicate sufficient time to learning when multiple courses need to be prepared at the same time. Finding the right areas of weakness can also be a challenge for students since the final quiz score may not accurately reflect the topics that need attention. A learner can score well on practice quizzes but still have difficulties in a certain topic or sub-topic. Past test papers can help with exam preparation, however, finding the same topics or questions in different papers typically requires manual comparison. The task of reviewing the multitude of these papers can be frustrating and confusing for students who may not be able to spot the recurring problems. Ultimately, separate management of diverse study activities is quite common among students. For instance, many students will use a particular source for their material, a different tool for their quizzes, and yet another tool for time management or monitoring their progress. Individually managing all these activities can make the whole study process less organized and make it harder for students to know what has already been learnt and what still needs to be done.

2.4 Goals and Objectives

The goal of AGAHI is to make university-level study and revision more organized and focused by bringing course material, practice, performance tracking, and revision support into a single platform. The objectives of the project are to:

- Allow students to upload course-related text-based PDFs, including lecture slides, notes, textbooks, and past papers.
- Process the uploaded material and use RAG to answer questions based on the student's own course

content.

- Provide source document and page references with answers where available.
- Record quiz performance at topic and sub-topic levels to identify weak areas.
- Use performance data to support adaptive revision, with greater focus on weaker topics while also revisiting previously covered topics.
- Create summaries, flashcards, and quizzes based on your uploaded content.
- Utilize performance data to adjust the revision process, with an emphasis on weaker topics along with previously covered topics.
- Provide the topic performance and advancement dashboard along with the weakness heatmap based on the topic.
- Evaluate past papers to find the prominent topics and questions.
- Implement study-support features like study streaks, prompt quiz feedback, and the customizable Pomodoro timer.

2.5 Project Scope

The scope of AGAHI is to develop a web-based academic study companion for university students, initially focusing on students at FAST-NUCES Lahore and similar educational environments in Pakistan. The system will support students in studying course material, practicing through quizzes, identifying weak areas, revising according to their performance, and managing their study sessions. The system will support the following:

- Uploading text-based PDF files such as lecture slides, notes, textbooks, and past papers.
- Processing and dividing uploaded documents into searchable sections.
- Asking questions and receiving answers based on the uploaded course material through RAG.
- Providing source and page references where available.
- Generating summaries, flashcards, and quizzes from course material.
- Tracking quiz performance at topic and sub-topic levels.
- Providing adaptive revision based on the student's performance.
- Showing progress and weak areas through a dashboard and heatmap.
- Analyzing past papers for recurring topics and question patterns.

- Providing study streaks, immediate feedback, and a customizable Pomodoro timer.

The project will use document processing, text chunking, embeddings, a vector database, and RAG for retrieving relevant course content. A separate database will store application data such as users, documents, quizzes, and learning progress. The final deliverable will be a functional web application containing the defined study, revision, assessment, and progress-tracking features. The scope has the following limitations: only text-based PDFs will be supported, with a maximum upload limit of 30 MB or 150 pages, whichever is reached first. Scanned or image-only PDFs, handwritten notes, and non-PDF files are not included. The system and document processing will be in English. The project will not include LMS integration, student enrollment or course registration, institutional administration, payment or financial services, or native Android/iOS applications. It will also not replace instructors or official university resources. Answers and generated study material will be based on the student's uploaded academic material rather than external web searches.

2.6 Sustainable Development Goal (SDG)



Figure 2.1: Quality Education
The figure represents SGD-4

Among the seventeen Sustainable Development Goals depicted in Figure 2.1, AGAHI is particularly coinciding with SDG 4, i.e., Quality Education, which refers to the aim of providing inclusive and equitable quality education for all. Among its targets, two targets are closely related to AGAHI's mission. According to Target 4.3, it ensures equal and affordable access to quality higher education, while the objective of Target 4.4 is to increase motivation for young individuals to acquire the necessary competencies for employment. Thus, AGAHI does not provide any new content in education and does not substitute teachers; instead, it makes learning more effective to the students.

The connection to SDG 4 comes from the problem itself. In our experience, a university student in Pakistan rarely fails a course because the material was unavailable. The slides, the textbook chapters and the past papers are usually all there. What is missing is guidance: knowing where to start, which topics deserve the most time, and whether one's understanding is actually improving. Students who can afford private tuition or paid preparation services get this guidance; many others do not. AGAHI takes the material a student already has and turns it into a structured study path, with summaries, flashcards, quizzes and a clear picture of weak areas. In doing so it offers a form of academic support that is not tied to a student's ability to pay for it, which is what "equitable" access means in practice.

The feature of inclusivity associated with Sustainable Development Goal No. 4 also impacted many of our design decisions. The tiny quiz sessions, the visible progress trackers, and the Pomodoro style focus timer were designed to take into account students who find it difficult to stay focused for extended periods of time, starting from those with Attention Deficit Hyperactivity Disorder. The tool designed to help students who don't have problems with sitting and reading for three hours won't be beneficial for those learners who need additional help. In addition, by basing all answers in the students' own coursework materials and citing the pages, AGAHI ensures students understand the curriculum rather than just copy and paste the knowledge, which is encouraged by regular chatbots.

2.7 Constraints

Every design decision in AGAHI was made under a set of constraints, some of which we chose deliberately to keep the project achievable and some of which are simply imposed on us by the environment we are building in. We list the important ones here so that the reader can judge the rest of this report against them.

- **Input format.** AGAHI only works with text-based PDF files. For the purposes of this system, scanned or image-based PDFs with no text layer, handwritten notes and non-PDF file formats such as Word documents or PowerPoint slides are not supported. Each file uploaded is limited to 30 MB or 150 pages, whichever comes first. These limitations are in place because the extraction, chunking and embedding of a single document needs to be processable by our cloud infrastructure within a reasonable amount of time.
- **Language.** The user interface and document processing pipeline only support the English language. Course material at FAST-NUCES and most Pakistani universities is in English, which covers our target audience, but also means that Urdu language notes or slides with a mix of languages will not be processed correctly.

- **Grounding.** All answers, summaries, flashcards and quiz questions must be generated from the documents the student has uploaded. AGAHI does not perform web searches and does not draw on general knowledge to fill gaps. This is a self-imposed constraint and it is central to the project: it is what makes the system trustworthy for exam preparation, but it also means that AGAHI can only be as good as the material a student gives it.
- **Dependence on third-party services.** The system relies on a hosted backend and database (Supabase), a vector store for embeddings, and an external provider for the language model and embedding models. We do not train or host our own models. Pricing, rate limits, latency and availability of these services are outside our control and directly affect the cost per student and the responsiveness of the application.
- **Time and team size.** The project is being delivered by a team of three students over two academic semesters, alongside regular coursework. This limits how many features can realistically be built, tested and polished, and it is the main reason that features such as LMS integration and support for non-text documents were placed out of scope from the beginning.
- **Quality of generated content.** Large language models have the potential to generate fluent but incorrect output. Even when grounded, a generated quiz question might be ambiguous or a summary might miss a nuance. The quality of everything AGAHI produces is also bounded by the quality of text extraction; a slide deck consisting mostly of diagrams will yield little usable text.
- **Data privacy and copyright.** Students will upload lecture slides and textbook excerpts that belong to instructors and publishers. Uploaded material is stored per user, is used only to serve that user, and is never shared between accounts or used to train models. The system is a personal study aid, not a content distribution platform, and we have to keep it that way.
- **Platform.** AGAHI is a web application intended for use on a laptop or desktop browser. A native mobile application is not part of the project.

2.8 Business Opportunity

AGAHI is first and foremost an academic project, and its main success criterion is whether it really does help students learn for their exams. That said, the problem it addresses is common enough that we think it is worth being honest about where a product like this could go.

The most obvious opportunity is the sheer size of the audience. Pakistan has a large and growing population of university students, almost all of whom deal with the same problem every semester: too much material, too little structure and not enough time. Existing options are poorly matched to them.

General purpose chatbots do not know the course, flashcard applications require the student to author everything by hand, and most of the polished study tools on the market are priced in US dollars for Western markets and are not built around the way courses are actually run here, with sessional exams, quizzes and heavy reliance on previous academic materials. A tool designed around the local academic workflow, priced for local students, has a clear gap to fill.

There are several ways such a product could sustain itself. A freemium based strategy is the most appealing: Some sort of free plan with a limit on monthly documents and quizzes can be uploaded and a cheap paid plan that raises those limits and unlocks features such as past-paper analysis or longer revision histories. Because the underlying language-model calls carry a real cost, the free tier also acts as a natural cap on expenses. A second route is institutional licensing, where a department or university provides AGAHI to its students and instructors gain, with appropriate consent, a view of which topics a class as a whole is struggling with. Finally, the most realistic go-to-market pathway is the one which does not cost a penny: starting with our own courses at FAST-NUCES Lahore, word of mouth and student societies to reach out to others, and refining the product on real students before considering anything wider.

We want to be clear that none of this is part of the FYP scope. What matters for this report is that the design does not close these doors: a per-user data model, a modular processing pipeline and a hosted backend are the same foundations a commercial version would need.

2.9 Stakeholders Description/ User Characteristics

A stakeholder is anyone who uses AGAHI, is affected by it, or has a say in whether it succeeds. Identifying them early mattered to us because several of our design decisions, such as strict grounding and the handling of uploaded material, are really responses to the concerns of a particular stakeholder. The people and groups involved are described below.

- **Students (primary users).** The primary users are the students. This covers the undergraduate and postgraduate population at FAST-NUCES Lahore as well as from other Pakistani universities. While they have no trouble with web applications or the like of a general AI chatbot, one would not expect them to have an understanding of the mechanics behind language or retrieval models. Study habits are not uniform; there are those who put in steady work over the course of the semester and then there are many who will do their revising in a more intensive fashion come exam time. Some, such as ADHD sufferers, find it hard to keep their attention for any length of time. They are to be found using the system in its entirety: to put up documents, pose questions, make use of the adaptive revision plan, try out quizzes and go over flashcards and summaries.

- **Instructors and TAs.** In the present scope they have no direct use for AGAHI, yet they can be considered the source of most of what is put through it; the system's priorities are set by their course objectives and old papers. What matters to them is that the tool serves to underpin the course as it is being taught and does not provide an end run around it.
- **FYP evaluation committee and project supervisor.** Ms. Sarah Abid Khan in her capacity as our supervisor will be setting the work's course, while the committee has to judge if the project has delivered on its objectives. They want to see a system that is well within its scope and technically sound, and one that is reported with honesty.
- **Development team.** The three of us design, build, test and document the system. We are also members of the target audience, which is useful for judging whether a feature actually helps, but it also means we have to be careful not to design only for our own study habits.
- **The institution.** FAST-NUCES, and potentially other universities, as a future adopter. The institution's concerns are academic integrity, the appropriate use of course material, and whether the tool improves learning outcomes rather than just grades.
- **Third-party service providers.** Supabase, the vector store, and the language-model and embedding providers on which the system runs. They are not users, but their pricing, terms of service and reliability directly shape what we can offer.

2.9.1 Stakeholders Summary

Table 2.1 summarises each stakeholder, their role in relation to AGAHI, and the nature of their involvement. Students are the only group that interacts with the system directly in the current scope; every other stakeholder either shapes the system's inputs, evaluates its outcomes, or provides the infrastructure it depends on. Keeping this distinction clear helped us prioritise: when a decision had to be made, the student's experience came first, followed by the concerns of instructors and the institution, with commercial considerations deliberately last.

2.9.2 Key High-Level Goals and Problems of Stakeholders

Each stakeholder comes to AGAHI with a goal and with a problem that stands in the way of that goal today. The paragraphs below explain the ones that most influenced the design, and Table 2.2 lays all of them out side by side.

The student's goal and problem are the reason the project exists, and they drove the two decisions we consider most important: that every generated artefact must be grounded in the student's own uploaded documents with a citation back to the source, and that quiz results must accumulate into a topic-level

Table 2.1: Summary of AGAHI stakeholders, their roles, and how they are involved with the system.

Stakeholder	Role	Involvement and Interest
Students	Primary end users	Upload course material, ask questions, study generated summaries and flashcards, attempt quizzes, follow the adaptive revision plan and track progress on the dashboard. Interested in preparing better in less time.
Instructors and teaching assistants	Indirect stakeholders; content authors	Their slides, notes and past papers are the raw material of the system. Interested in the tool reinforcing course objectives and not encouraging shortcuts.
Supervisor and evaluation committee	Guidance and assessment	Review scope, design, implementation and reporting. Interested in a technically sound project that meets its objectives.
Development team	Designers and builders	Responsible for delivering, testing and documenting the system within two semesters.
Institution (FAST-NUCES and others)	Potential adopter	Interested in academic integrity, appropriate handling of course material, and measurable learning benefit.
Third-party service providers	Infrastructure	Supabase, the vector store and the model providers. Their limits and pricing constrain performance and cost.

picture of strengths and weaknesses instead of being thrown away after each attempt. The 70/30 split in the adaptive revision engine follows directly from this; it is our answer to the student who does not know what to study next.

The concerns of instructors and the institution reinforced the grounding decision from a different direction. A tool that answers from the course material, in the course's own terminology, and refuses to reach outside it, is far easier to defend as a legitimate study aid than one that draws on the open web. The evaluation committee's concern about over-promising is why the constraints in Section 2.7 are stated as plainly as they are, and why features that we could not deliver responsibly in two semesters were excluded from scope rather than left vague. Finally, our own goal as the development team, to build something we would actually use, is a useful check on every feature: if we would not open it the night before an exam, it probably does not belong in the product.

Table 2.2: High-level goals of each stakeholder and the problems they currently face that AGAHI aims to address.

Stakeholder	High-Level Goal	Current Problem
Students	Prepare effectively for exams by focusing effort where it is needed most, and be able to trust the answers they get.	Material is scattered across many files; no reliable way to know which topics are weak; general chatbots give answers that cannot be traced back to the course; long study sessions are hard to sustain.
Instructors and teaching assistants	Students who engage with the course as taught and arrive at exams with a genuine understanding of it.	Students often revise from unverified sources or generic AI output that uses different terminology or contradicts the course.
Supervisor and evaluation committee	A project that is well scoped, technically defensible and delivered on time.	FYPs built on language models risk over-promising; grounding, evaluation and scope must be handled carefully.
Development team	Deliver a working, useful system within two semesters and learn from doing so.	Limited time, dependence on external services with real costs, and the difficulty of evaluating generated content objectively.
Institution	Better learning outcomes without compromising academic integrity or misusing course material.	Existing AI tools are hard to govern and are not built around institutional course structures.
Third-party service providers	Sustained, compliant use of their platforms.	Not applicable in the usual sense; their role is to supply infrastructure, and it is their rate limits and pricing that pose a problem for us rather than the other way round.

Chapter 3 Literature Review / Related Work

AGAHI unifies two streams of research that are traditionally studied independently: the design of study tools that students actively use, and the design of language models that answer grounded in a fixed set of documents. To make this approach practical, we examined both fields. This chapter reviews the literature that informed our design decisions. We evaluate existing study applications to identify what students currently utilize and where current solutions fall short for course-specific examination preparation. Furthermore, we analyze the theoretical and empirical work surrounding Retrieval-Augmented Generation (RAG), automated question generation, and the cognitive science that motivates features such as flashcards, adaptive revision, and focused study intervals. Every related work is summarized, critically analyzed, and related back to AGAHI. Summary tables and a concluding overview synthesize the findings at the end of the chapter.

3.1 Definitions, Acronyms, and Abbreviations

The following terms are used throughout this chapter. The meanings defined in this section apply consistently across the rest of the report.

LLM (Large Language Model): A deep neural network trained on massive volumes of text capable of producing fluent natural language in response to user prompts. AGAHI uses a hosted LLM to generate answers, summaries, flashcards, and quiz questions.

RAG (Retrieval-Augmented Generation): A technique where relevant passages are retrieved from an external document corpus and supplied as context to a language model, ensuring that the generated output derives from the retrieved passages rather than the model's parametric memory alone.

Retriever: The component of a RAG pipeline responsible for identifying and selecting relevant text passages corresponding to a user query.

Generator: The component of a RAG pipeline, typically a pre-trained language model, that synthesizes the final response from the query and the retrieved context passages.

Embedding: A fixed-length vector of real numbers representing the semantic meaning of a text segment, produced by an embedding model. Texts with similar meanings yield vectors that are geometrically close in vector space.

Vector Database (Vector Store): A specialized database engineered to store vector embeddings and perform high-dimensional similarity searches efficiently to return records closest to a query vector.

Chunking: The process of partitioning a long document into smaller, coherent passages (chunks) that can be embedded and indexed independently.

Top-k Retrieval: A retrieval strategy that selects the k most similar chunks relative to a query vector,

where k is a small predefined integer such as 5 or 10.

DPR (Dense Passage Retrieval): A retrieval framework that encodes queries and document passages into dense vectors using two separate BERT-based encoders and identifies matches via dense vector similarity.

MIPS (Maximum Inner Product Search): The mathematical search problem of finding stored vectors that maximize the inner product with a query vector, which forms the computational basis of vector database retrieval.

Parametric Memory: Knowledge stored implicitly within the learned weights and parameters of a neural network model.

Non-Parametric Memory: Knowledge stored explicitly outside model parameters, such as in an external document index or vector store, which can be modified without retraining the model.

Grounding: Constraining a language model's output to strictly reflect facts asserted in the supplied source material. In AGAHI, generated content must derive exclusively from the student's uploaded documents.

Context-Only Prompting: Prompt engineering that directs the language model to answer solely using the passages provided in the prompt and to explicitly state when the source passages do not contain the necessary information.

Hallucination: The phenomenon wherein a language model generates fluent, authoritative-sounding text that is factually inaccurate or unsupported by the provided reference context.

Open-Domain Question Answering: The task of answering factual questions where the relevant source documents are not specified in advance and must be identified dynamically by the system.

Seq2seq (Sequence-to-Sequence) Model: A neural network architecture that maps an input sequence of tokens to an output sequence of tokens, utilized as the generator in early RAG architectures.

BART: A pre-trained sequence-to-sequence model developed by Facebook AI Research that combines bidirectional and autoregressive transformers, used as the generator in the foundational RAG paper.

Flashcard: A two-sided study artifact featuring a question or cue on the front and the corresponding answer on the back, utilized for active self-testing.

Active Recall: The learning practice of actively retrieving information from memory, such as through self-quizzing, rather than passively re-reading study notes.

Testing Effect: The cognitive phenomenon where being tested on material produces superior long-term memory retention compared to spending an equivalent amount of time re-reading it.

Spaced Repetition: A study methodology that schedules reviews of learned material at increasing time intervals to optimize memory retention and interrupt the forgetting curve.

Distractor: An incorrect but plausible alternative option provided alongside the correct answer in a

multiple-choice question.

Gamification: The integration of game mechanics, such as streaks, points, and timed milestones, into non-game contexts to sustain user motivation and engagement.

Pomodoro Technique: A time-management method that structures work into focused intervals (traditionally twenty-five minutes) separated by short five-minute breaks.

EFL (English as a Foreign Language): The study of English by non-native speakers within educational contexts where English is not the primary language of daily interaction.

ADHD (Attention Deficit Hyperactivity Disorder): A neurodevelopmental condition characterized by persistent difficulties with sustained attention, task initiation, and focus over extended periods.

LMS (Learning Management System): Institutional software platforms used to distribute academic material, track course progress, and administer evaluations, such as Google Classroom or Moodle.

PDF (Portable Document Format): The primary document standard accepted by AGAHI. A text-based PDF contains a digitally extractable character layer, whereas an image-based PDF consists solely of scanned page bitmaps.

3.2 Detailed Literature Review

The work relevant to AGAHI falls into a few natural groups, and we have organised this section around them. The first group is *existing study tools*: the flashcard, note-taking and "chat with your PDF" applications that students already use. Reviewing these told us what is already solved and, more importantly, what these tools do not do for a student preparing for a specific course. The second group is *research on retrieval-augmented generation and grounded question answering*, which is the technical foundation of AGAHI's question-answering, summarisation and quiz-generation features. The third group covers *automatic question generation for education*, since generating good quiz questions and distractors from a document is a research problem in its own right. The final group is *learning science*: the evidence on testing, spaced practice and engagement that justifies why AGAHI has flashcards, an adaptive revision plan and short gamified sessions at all, rather than being just a chatbot.

For each item we follow the same structure: a summary of what the work is and what it found, a critical analysis of its strengths and weaknesses, and a discussion of how it relates to AGAHI. We have tried to be even-handed. Several of the tools reviewed here are excellent at what they do; our interest is in the specific gap between what they offer and what a university student in Pakistan needs the week before an exam.

3.2.1 Quizlet: Flashcard-Based Study Platform

Quizlet [1] is one of the most widely used study applications in the world and the first tool most students mention when asked how they revise definitions and vocabulary. We reviewed it because it represents the mature, well-tested end of the "self-testing" approach that AGAHI's flashcard and quiz features build on, and because there is published research on its effect on learning.

3.2.1.1 Summary

Quizlet is a web and mobile platform built around user-created *study sets*: lists of term–definition pairs that can be studied in several modes. The core modes are Flashcards, for simple two-sided review; Learn, which mixes flashcards, multiple-choice and written-answer questions and adapts which items it shows based on the learner's answers; Test, which produces a practice test from the set; and Match, a timed game in which the learner pairs terms with their definitions as quickly as possible. Sets can be kept private or shared, and the public library of sets is very large, so for many standard courses a student can find a ready-made set instead of writing one. The platform runs on a freemium model, with a paid tier that removes advertisements and unlocks additional features. In recent years Quizlet has added AI-assisted features, including a conversational tutor that quizzes the learner on a set and a tool that generates study material from a learner's pasted notes.

The research evidence on Quizlet is largely positive within its niche. Waluyo and Bucol [2] studied gamified vocabulary learning with Quizlet among low-proficiency English-language learners and reported improvements in vocabulary knowledge alongside favourable learner attitudes towards the tool. Their study is one of a number of classroom studies that reach a similar conclusion: when the material is naturally decomposable into discrete items that must be memorised, Quizlet's combination of self-testing and game-like modes works, and learners tend to enjoy using it.

3.2.1.2 Critical Analysis (Strengths and Weaknesses)

Quizlet's strengths are real and worth learning from. The barrier to entry is almost zero; a student can create a set and start studying within minutes. The study modes are well designed and cover the spectrum from passive review to active recall under time pressure, and the Match game in particular shows how a small amount of gamification can make repetitive review tolerable. The Learn mode's habit of returning to items the learner got wrong is a simple form of adaptivity that students respond to. The public library reduces the effort of set creation for common courses, and the evidence from studies such as [2] shows measurable learning gains for the kind of material Quizlet is designed for.

The weaknesses become apparent as soon as the material stops being a list of terms. The fundamental

unit is an atomic term–definition pair, which suits vocabulary, formulas and named concepts but suits multi-step reasoning, worked examples and "compare X with Y" questions poorly; a good deal of what appears in a university exam does not fit on one side of a card. Content creation is the student's responsibility: either they write the set by hand, which takes the very time they are short of, or they rely on a public set of unknown accuracy written for a different instructor's version of the course. The newer AI features generate items from a single pasted note rather than from the whole body of a course's material, and nothing generated links back to a page in the source, so a student cannot easily verify a card against their own slides. Progress is tracked per set rather than per topic across a course, so Quizlet cannot tell a student that they are weak on, say, normalisation in a database course when the relevant cards are spread across three sets. There is no notion of past papers at all. Finally, several of the more useful features sit behind the paid tier, which is priced for Western markets.

3.2.1.3 Relationship to the Proposed Work

Quizlet validated two of our design choices and defined the gap AGAHI is meant to fill. It validated that flashcards and short, game-like sessions are things students will actually use, and the classroom evidence [2] gave us confidence that self-testing in this form improves retention rather than just feeling productive. AGAHI's flashcard mode, quiz sessions with immediate feedback, and streak-based motivation are all recognisably in the same family.

The difference lies in where the content comes from and what the system remembers. AGAHI generates flashcards and quiz questions from the student's own uploaded course documents, with a reference to the source page, rather than asking the student to author them or trust a stranger's set. It organises everything by topic and sub-topic extracted from the course material, so that quiz performance builds into a course-level picture of weak areas instead of a per-set score. That topic-level picture is what makes the 70/30 adaptive revision plan and the weakness heatmap possible, and neither has an equivalent in Quizlet. Past-paper analysis is likewise entirely outside Quizlet's model. In short, AGAHI keeps what Quizlet gets right about the act of studying and replaces the parts that make a student do the preparation work by hand.

3.2.2 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)

The paper by Lewis et al. [3] introduced the term Retrieval-Augmented Generation and the general architecture that AGAHI's question-answering, summarisation and quiz-generation features are built on. Understanding what the original work did, and where its assumptions differ from ours, was essential before we committed to a RAG-based design.

3.2.2.1 Summary

The authors start from an observation about pre-trained language models: they store a great deal of factual knowledge in their parameters, but that knowledge is difficult to inspect, difficult to update and unreliable on knowledge-intensive tasks such as open-domain question answering. Their proposal is to combine two kinds of memory. The *parametric* memory is a pre-trained seq2seq generator, BART, whose weights encode general language ability. The *non-parametric* memory is a dense vector index of Wikipedia, in which the December 2018 dump was split into roughly 21 million passages of 100 words each and encoded with the document encoder from Dense Passage Retrieval (DPR) [4]. At inference time a DPR query encoder embeds the input, a maximum inner product search over the index returns the top-k passages (with k typically 5 or 10), and the generator conditions on both the input and the retrieved passages to produce the output.

Two variants are proposed. RAG-Sequence retrieves once and uses the same set of passages to generate the entire output, marginalising over passages at the sequence level. RAG-Token allows a different retrieved passage to inform each generated token, which lets the model draw on several passages within a single answer. The retriever and generator are trained jointly end-to-end, but only the query encoder and the generator are fine-tuned; the document encoder and the index are kept fixed, which avoids the cost of re-indexing during training. The models were evaluated on four open-domain question answering datasets (Natural Questions, TriviaQA, WebQuestions and CuratedTrec), where they set new state-of-the-art results among comparable systems, and additionally on abstractive question answering (MS MARCO), Jeopardy-style question generation and fact verification (FEVER). Human evaluators found RAG's generations to be more factual and more specific than those of the BART baseline. The paper also demonstrates that the non-parametric memory can simply be swapped, replacing the 2018 Wikipedia index with a 2016 one, to change what the model "knows" without any retraining.

3.2.2.2 Critical Analysis (Strengths and Weaknesses)

The paper's central strength is that it offers a general, clean architecture rather than a task-specific trick. Separating knowledge from language ability gives three practical benefits that matter enormously for an educational tool: the source of an answer can be inspected, because the retrieved passages are visible; knowledge can be updated by changing the index rather than the model; and the generator is pushed towards the retrieved evidence rather than towards whatever it happened to memorise, which the human evaluation confirms reduces factual errors. The evaluation is broad, spanning extractive and abstractive QA, generation and classification, and the index-swap experiment is a simple and convincing demonstration of the approach's flexibility. The DPR retriever, and the finding that fine-tuning only the

query encoder is sufficient, have been very influential; most practical RAG systems today follow the same dense-retrieval pattern [5].

The limitations mostly stem from the setting the work was evaluated in. The knowledge source is Wikipedia, a very large and generally well-written corpus of general knowledge, and the tasks are short factual questions. Neither resembles a set of lecture slides with terse bullet points, a textbook chapter with worked examples, and a scanned-then-OCR'd past paper. Retrieval quality is the bottleneck of the whole system, and the paper offers little guidance on what happens when the corpus is small and noisy. The fixed 100-word chunking discards document structure entirely, so there is no notion of which page, section or slide a passage came from, which makes precise citation impossible without additional bookkeeping. The model has no explicit mechanism for declining to answer when the retrieved passages do not contain the answer; it will generate something regardless, which is exactly the failure mode a student cannot afford. End-to-end fine-tuning of the retriever and generator requires substantial compute and labelled question–answer pairs, which is out of reach for most small teams. Finally, the generator is a 2020-era seq2seq model, and the authors could not anticipate the shift towards using large frozen LLMs with retrieved context supplied in the prompt, which is now the common practice and changes several of the engineering trade-offs.

3.2.2.3 Relationship to the Proposed Work

AGAHI's core question-answering feature is a direct descendant of this architecture. The student's uploaded course PDFs play the role Wikipedia plays in the paper: they are the non-parametric memory. Each document is split into chunks, each chunk is embedded and stored in a vector database, a question is embedded with the same model, the top-k most similar chunks are retrieved, and a language model generates an answer from the question and those chunks. Summarisation, flashcard generation and quiz generation follow the same retrieve-then-generate pattern, differing only in what is asked of the generator. The paper's key argument, that grounding generation in retrieved evidence produces more factual and more specific output, is precisely the property AGAHI needs in order to be trusted for exam preparation.

Where AGAHI departs from the original work is in the places where its limitations bite. We do not fine-tune anything; we use a hosted LLM as a frozen generator and supply the retrieved chunks in the prompt, which is far cheaper and lets the generator be upgraded without touching the pipeline. We keep document, page and section metadata alongside every chunk so that answers can cite exactly where they came from, which the paper's 100-word chunks could not do. Most importantly, we impose a constraint the paper does not: the generator is instructed to answer only from the retrieved context and to say so when the context does not contain the answer, rather than falling back on parametric knowledge. Lewis

et al. treated the two memories as complementary; for a course-aware study tool the parametric memory is the thing to be wary of, because it is where terminology from a different textbook and confident but off-syllabus explanations come from. The paper's own finding that retrieval quality determines everything is also why chunking strategy and topic extraction receive as much attention in our design as the language model does.

3.2.3 ChatPDF: Conversational Question Answering over a Single Document

ChatPDF [6] represents the most visible commercial application among document-grounded conversational systems. For many undergraduates, this tool provided their initial exposure to language models answering queries directly from user-uploaded files. We evaluated the platform because its core retrieval-augmented pipeline closely mirrors AGAHI's isolated question-answering subsystem. It demonstrates competent document-grounded retrieval. Yet it stops there.

3.2.3.1 Summary

ChatPDF provides a minimalist web workflow. The user uploads a PDF file, the server returns an automated introductory summary, and the interface opens a split-screen workspace featuring the PDF viewer alongside a chat interface. Generated responses include interactive citations. Clicking a citation jumps the document viewer directly to the source excerpt, allowing the user to verify facts against the original text immediately. Friction remains minimal. The public tier functions without user authentication, permitting two uploads daily capped at 120 pages and 10 MB alongside a fixed daily query quota. A low-cost subscription tier expands these boundaries to 2,000 pages per file, raises daily prompt allocations, and unlocks multi-document folders for cross-file queries. The platform also exposes an external developer API that reports the specific page numbers utilized during generation. This confirms that page-level provenance represents an architectural priority rather than an aesthetic add-on.

3.2.3.2 Critical Analysis (Strengths and Weaknesses)

The platform succeeds primarily because it implements observable grounding. Placing the source document and conversational thread within a single display viewport, while converting raw citations into direct hyperlinks, replaces blind trust with instant empirical verification. This capability separates dedicated educational utilities from ungrounded conversational agents. ChatPDF proves that this retrieval workflow operates effectively inside an interface requiring zero onboarding. Its frictionless entry, requiring no account creation, illustrates how simple document ingestion can become.

Major shortcomings stem from its narrow identity as an isolated question-answering utility rather than a pedagogical system. The application answers discrete questions and halts immediately. It generates no

flashcards, provisions no diagnostic quizzes, logs no telemetry regarding student comprehension, and retains no historical continuity across study sessions. Every conversational session remains bound to a solitary file or static folder rather than an academic course syllabus. The system lacks any underlying topic hierarchy to connect themes across weeks of lecture slides; consequently, it cannot detect persistent conceptual confusion or prompt targeted remediation. Past exam papers remain indistinguishable from generic slide decks. The free tier's page cap excludes standard university textbooks, which penalizes students with limited financial resources. Fast answers do not guarantee learning. Kasneci et al. [7] observed that conversational fluency often encourages passive answer collection over conceptual mastery unless the software deliberately redirects students toward active testing and reflection. ChatPDF never attempts such pedagogical intervention.

3.2.3.3 Relationship to the Proposed Work

ChatPDF substantiated our primary architectural premise before implementation began: undergraduates value grounded responses that explicitly link claims to source pages. AGAHI adopts this proven paradigm. Our retrieval module replicates page-level source attribution and direct document navigation. However, AGAHI treats question answering as an initial step within an integrated learning cycle rather than a complete terminal product. Questions exist inside a broader course scope rather than an isolated file container. The same topic taxonomy that organizes lecture slides tags incoming student questions. Generated summaries, flashcards, and diagnostic quizzes reside within that unified workspace. Historical evaluation data drives adaptation. Cumulative quiz scores expose persistent topic deficiencies that feed into the adaptive study schedule. ChatPDF demonstrates how to ground isolated responses reliably; AGAHI focuses on what the student should do once that answer is delivered.

3.2.4 Turbo AI (formerly Turbolearn AI): An Integrated Note, Flashcard and Quiz Generator

Turbo AI [8], which operated as Turbolearn AI prior to its rebranding, generates notes, flashcards, and quizzes from student materials. The platform shares considerable functional overlap with AGAHI. We analyzed it closely for this reason. Examining Turbo AI reveals existing student expectations and highlights areas where a course-aware platform offers superior pedagogical value.

3.2.4.1 Summary

Turbo AI ingests a broad array of multimodal formats. The platform processes uploaded PDF files, YouTube URLs, audio recordings, video files, and live lecture audio captured directly through its mobile or web interface. From these inputs, the system synthesizes structured, editable lecture notes complete

with section headings, tabular comparisons, diagrams, and equations. Audio recordings feature interactive timestamps mapped directly back to source media segments. The system subsequently generates spaced-repetition flashcards to highlight user retention deficits, alongside customizable quizzes where learners adjust topic scope and item difficulty. Students can also convert lecture notes into podcast-style audio summaries or interact conversationally with the underlying model. The platform organizes assets into hierarchical folders, provides collaborative real-time editing, and synchronizes state across client devices. Beyond personal uploads, Turbo AI offers pre-authored study guides for United States Advanced Placement curricula and selected academic textbooks under a freemium commercial model.

3.2.4.2 Critical Analysis (Strengths and Weaknesses)

Turbo AI builds a cohesive study loop. By converting raw multi-format media into structured notes, spaced-repetition cards, and diagnostic quizzes within a solitary interface, it establishes self-testing as an expected baseline rather than a novel feature. Multimodal ingestion functions reliably. The live audio transcription engine exceeds AGAHI's planned capabilities by a wide margin, and providing fully editable notes actively encourages student engagement through manual corrections. Public reviews reflect broad user satisfaction regarding note fidelity and flashcard coherence.

Architectural limitations emerge upon closer inspection. The system structures data around isolated individual notes derived from discrete uploads. While folders group files logically, the platform lacks a cross-document topic ontology capable of mapping student competencies across a semester-long course syllabus. Topic tracking operates strictly within single notes. Consequently, learners cannot observe macro-level course mastery. The interface provides no page-level provenance. When students doubt a generated quiz option or summary statement, the system offers no direct citation link back to the source text. Past examination papers receive no specialized treatment. Commercial content aligns with North American secondary schooling, and pricing remains dollar-denominated. Multimodal ingestion carries substantial operational volatility. Multiple user reports document dropped audio streams and truncated transcriptions during extended lectures, demonstrating the severe engineering overhead required to maintain real-time media processing.

3.2.4.3 Relationship to the Proposed Work

Turbo AI proves that undergraduates demand an integrated revision ecosystem instead of fragmented single-purpose tools. It defines the functional baseline that AGAHI must satisfy. Our team chose not to pursue multimodal capture. Restricting AGAHI to text-based PDF documents represents a deliberate engineering choice to maximize extraction stability within a two-semester undergraduate timeline. AGAHI differentiates itself structurally. Our architecture organizes all materials around a unified course

topic hierarchy derived across multiple documents simultaneously, tracking student mastery per concept rather than per note. Every synthesized artifact includes verifiable page-level citations tied directly to uploaded texts. In addition, AGAHI treats past examination papers as primary analytical inputs, extracting recurring exam themes to drive our 70/30 adaptive revision algorithm. Integrated Pomodoro focus timers directly support learners who struggle with sustained attention. Finally, AGAHI addresses local educational workflows at FAST-NUCES and peer institutions rather than foreign curricula. Turbo AI validates our high-level functional model; AGAHI delivers that model with course-level awareness, verifiable attribution, and local relevance.

3.2.5 A Systematic Review of Automatic Question Generation for Educational Purposes

Generating high-quality academic evaluation items algorithmically presents severe technical hurdles. Kurdi et al. [9] published an extensive systematic review of automatic question generation (AQG) in the *International Journal of Artificial Intelligence in Education*. We examined this survey because AGAHI's automated quiz and flashcard generation modules encounter the exact design dilemmas that the authors map out.

3.2.5.1 Summary

The authors synthesized AQG literature published between 2015 and early 2019, analyzing 93 papers organized into 72 primary studies. The review classifies each project across core parameters: pedagogical intent, target domain, input data representation, question formats, algorithmic generation methods, and evaluation protocols. Manual question writing demands extensive domain expertise and consumes substantial instructor time. This scarcity directly bottlenecks adaptive testing systems that require deep question pools.

The findings highlight critical technical gaps. Most surveyed systems failed to control question difficulty systematically, rarely diversified question formats, neglected template automation, and largely omitted explanatory feedback for incorrect student attempts. For multiple-choice assessments, generating viable distractors represents the central engineering bottleneck. The survey outlines several distractor generation approaches: word-frequency filtering, part-of-speech matching, semantic similarity thresholds, knowledge-graph ontology traversals, query relaxation techniques, and machine learning ranking models. Evaluation methodologies across the surveyed literature remain fractured. Few studies apply standardized empirical metrics, creating a pressing need for reproducible evaluation frameworks that balance feasibility with diagnostic rigor.

3.2.5.2 Critical Analysis (Strengths and Weaknesses)

The review excels in methodological rigor. The literature search criteria remain fully transparent, the structural taxonomy applies consistently across diverse projects, and the resulting classification provides an objective map of prior algorithmic successes and failures. The authors isolate distractor construction as the decisive factor determining multiple-choice question efficacy. Their critique regarding neglected difficulty calibration and missing learner feedback remains accurate today.

Temporal constraints limit the review's immediate applicability. The surveyed research predates modern autoregressive transformer architectures. Consequently, rule-based syntactic transformations and template pipelines have largely yielded to direct language model prompting. Nevertheless, underlying pedagogical challenges persist across architectural shifts. Modern foundation models frequently generate multiple-choice questions with erratic difficulty tiers and trivial distractors unless strictly constrained. Automated text generation has widened the evaluation deficit. The survey also concentrates on standalone laboratory prototypes rather than production software deployed over full academic semesters. Longitudinal factors like repetitive question fatigue and alignment with official examination rubrics receive little attention. Finally, analyzed datasets lean heavily toward structured linguistics and ontology-rich domains rather than unstructured academic slide presentations.

3.2.5.3 Relationship to the Proposed Work

AGAHI constructs assessment items directly from unstructured text documents, confronting the most difficult input modality identified by Kurdi et al. Distractor validity directly impacts our platform. Our engine extracts distractors exclusively from related yet distinct concepts within the student's uploaded course documents, producing plausible alternatives that probe common conceptual confusions rather than trivial variations. The survey's warnings regarding uncalibrated difficulty directly inform our adaptive revision algorithm. The 70/30 question distribution guarantees that students receive targeted remediation on deficient topics alongside balanced retention testing. In addition, AGAHI pairs every question with explicit explanatory rationale and source page citations, transforming failed quiz attempts into actionable study prompts. Methodological criticisms raised in the review provide clear guardrails for our second phase. Evaluating AGAHI's generated questions will require formal validation protocols assessing conceptual relevance, factual correctness, linguistic clarity, and distractor plausibility rather than casual inspection.

3.3 Literature Review Summary Table

The three tables below condense the review into one line per work, following the grouping used throughout this chapter. Table 3.1 covers the study tools and products that students already use, Table 3.2 covers the research on retrieval-augmented generation and grounded answering, and Table 3.3 covers automatic question generation and the learning science behind self-testing and spaced review. Besides the five works given full treatment in Section 3.2, the tables include tools and papers that we consulted while designing AGAHI but did not have room to review at length: Anki, Gemini Notebook and Notion AI among the products, and the dense retrieval, sentence embedding, domain adaptation, hallucination and learning-science papers among the research. In every table the final column is written relative to AGAHI's goal of course-level, verifiable exam preparation; it is not a general judgement of the work's quality.

Table 3.1: Summary of existing study tools and products reviewed

Application	Key features	Relevance to AGAHI	Limitations (relative to AGAHI's goals)
Quizlet [1]	User-created study sets; Flashcards, Learn, Test and Match modes; large public set library; freemium; AI features (Q-Chat, Magic Notes) added in 2023	Validates self-testing and short game-like sessions; direct model for AGAHI's flashcard mode and streaks	Content is item-based and user-typed, not drawn from the student's own course documents; no grounding or citations; no course-level topic mastery or past-paper view
Anki [10]	Open-source flashcards; SM-2 and FSRS spaced-repetition schedulers with a per-card memory model; shared decks and add-ons	Reference point for how review should be scheduled and for weighting weak items, which AGAHI's 70/30 plan applies at topic level	Cards must be authored by hand; steep setup; no document grounding, no generation, no course-level analytics
ChatPDF [6]	Chat over a single uploaded PDF (folders on paid tier); answers carry clickable page citations; API returns page numbers; free tier limited to 2 PDFs/day, 120 pages, 10 MB	Closest product to AGAHI's grounded question answering; proves page-level provenance is feasible and valued by users	Question answering only: no flashcards, quizzes or mastery tracking; scope is a document, not a course; free page limit excludes most textbooks
Gemini Notebook (formerly NotebookLM) [11]	Source-grounded notebook; answers cite passages in uploaded sources; up to 50 sources per notebook on the free tier; Audio Overviews; flashcard generation	Closest in spirit to multi-document grounding with citations; shows that a grounded assistant can span many files	General research tool, not organised around a course or exam; no topic mastery, adaptive revision or past-paper analysis; tied to the Google ecosystem
Notion AI [12]	Assistant inside the Notion workspace: question answering across pages and uploaded PDFs with citations, writing help, meeting notes; full access on paid Business plan	Shows that students already keep notes in general-purpose tools and want to question them	Built for workplace knowledge, not study; no quizzes, flashcards or mastery tracking; full AI access is paid
Turbo AI (formerly Turbolearn AI) [8]	Notes, flashcards and quizzes generated from PDFs, YouTube links, audio and live lecture recordings; spaced-repetition flashcards with weak-area tracking; quiz topic and difficulty selection; folders and sharing	Closest overall competitor; confirms demand for the full material-to-practice loop and that adaptive review is now expected rather than distinctive	Progress and weak areas are tracked per note, not across a course's documents; no past-paper recurrence analysis; study-guide library is US/AP-oriented; breadth of inputs beyond AGAHI's scope

Table 3.2: Summary of research on retrieval-augmented generation and grounded answering

Work	**Method / contribution**	**Relevance to AGAHI**	**Limitations (relative to AGAHI's goals)**
Lewis et al. (2020) [3]	Retrieval-Augmented Generation: DPR retriever over 21M Wikipedia passages plus BART generator; RAG-Sequence and RAG-Token; joint fine-tuning; index swap	Architectural basis for AGAHI's question answering, summarisation and quiz generation	Open-domain Wikipedia setting; 100-word chunks discard document structure; generator can still answer from parametric memory when retrieval fails
Karpukhin et al. (2020) [4]	Dense Passage Retrieval: dual BERT encoders trained on question–passage pairs; outperforms BM25 on open-domain QA	The dense-retrieval pattern AGAHI uses to find relevant chunks for a query	Trained on general QA data; retrieval quality degrades on domain-specific terminology without adaptation
Reimers and Gurevych (2019) [13]	Sentence-BERT: siamese BERT producing fixed-size sentence embeddings comparable by cosine similarity; large speed-up for similarity search	Underpins the sentence-embedding models used to index and search course chunks	Sentence-level model; quality depends on training domain; long chunks must be split or truncated
Gao et al. (2023) [5]	Survey of RAG for LLMs: naive, advanced and modular RAG; chunking, reranking, query rewriting; evaluation practice	Practical guide to AGAHI's pipeline choices (chunk size, top-k, prompting, evaluation)	Survey without new experiments; field moves quickly; no education-specific guidance
Siriwardhana et al. (2023) [14]	RAG-end2end: jointly adapts retriever and generator to a new domain with an auxiliary reconstruction signal; evaluated on specialised domains	Evidence that domain shift hurts retrieval, motivating careful chunking and context-only prompting for course material	Requires fine-tuning and domain data that AGAHI cannot afford; not evaluated on educational material
Ji et al. (2023) [15]	Survey of hallucination in NLG: intrinsic and extrinsic types, metrics and mitigation across tasks including generative QA and summarisation	Defines the failure mode that AGAHI's grounding and citations are meant to control; informs how we will evaluate faithfulness	Largely pre-LLM-era studies; mitigation methods are task-specific; no education-specific evaluation

Table 3.3: Summary of research on automatic question generation and learning science

Work	**Method / contribution**	**Relevance to AGAHI**	**Limitations (relative to AGAHI's goals)**
Kurdi et al. (2020) [9]	Systematic review of 93 AQG papers (2015 to early 2019): inputs, question forms, generation methods, distractor strategies, evaluation	Checklist for AGAHI's quiz generator: distractor quality, difficulty control, feedback and a stated evaluation protocol	Pre-dates LLM-based generation; documents inconsistent evaluation practice without resolving it
Kasneci et al. (2023) [7]	Position paper by education and AI researchers on opportunities and risks of LLMs for learners and teachers	Frames the risks (overreliance, bias, hallucination) that motivate traceable answers and a practice-first design	Discussion rather than empirical evidence; written early in the ChatGPT era
Roediger and Karpicke (2006) [16]	Two experiments: students tested on prose passages retained more after 2 days and 1 week than students who re-studied	Empirical basis for making quizzes and flashcards central to AGAHI rather than optional extras	Laboratory setting with prose passages; no feedback given; retention measured over at most one week
Cepeda et al. (2006) [17]	Meta-analysis of 317 experiments on distributed practice; optimal spacing between sessions grows with the retention interval	Justifies spacing review sessions and returning to weak topics over time rather than cramming	Verbal recall tasks, mostly in laboratories; does not prescribe schedules for complex course material
Dunlosky et al. (2013) [18]	Review of ten learning techniques; practice testing and distributed practice rated high utility, rereading and highlighting low	Supports steering students from passive reading towards practice, which is what AGAHI's loop does	Narrative review; effects vary by material and learner; little evidence specific to digital tools
Waluyo and Bucol (2021) [2]	Classroom study of gamified vocabulary learning with Quizlet among low-proficiency learners; vocabulary gains and positive attitudes	Evidence that flashcard self-testing and light gamification work in a real classroom	Vocabulary learning in one context; does not generalise directly to conceptual university courses

3.4 Conclusion

Reading across the tools and research reviewed in this chapter, a consistent picture emerges. Each of the existing products does one part of the job well. Quizlet [1] shows that self-testing with flashcards and short game-like sessions works and that students will use it, with classroom evidence to back that up [2]. ChatPDF [6] shows that answering questions from a student's own document, with a citation that can be checked in one click, is both feasible and valued. Turbo AI [8] shows that students want the whole loop, from material to notes to flashcards to quizzes, in a single place, and that adaptive review has become an expected feature. None of them, however, is organised around a *course*, and the same holds for the other tools in Table 3.1. They operate on a study set, a deck, a file, a notebook or a note, and so none of them can tell a student which topics across a semester's worth of material are weak, how those topics relate to what the course has historically examined, or what to study next and in what proportion. That is the gap AGAHI is built to fill, and the summary tables in the preceding section record the same conclusion work by work.

The research literature supplied both the method and the warnings. Lewis et al. [3] established retrieval-augmented generation as the architecture for making a language model answer from a fixed body of documents, and the practical consensus that has grown up around it [5, 4] is what AGAHI's ingestion, retrieval and generation pipeline follows. The same work makes clear that retrieval quality is the limiting factor and that a model will still answer when the retrieved passages do not support an answer, which is why AGAHI keeps page-level metadata on every chunk and instructs the generator to stay within the retrieved context. Kurdi et al. [9] mapped the state of automatic question generation and found that difficulty control, distractor quality, feedback and rigorous evaluation are routinely neglected; those four items are, in effect, a checklist for AGAHI's quiz generator and for how we will evaluate it in the second phase of the project. Kasneci et al. [7] added the broader caution that language models in education can just as easily shortcut learning as support it, which reinforces our decision to make every answer traceable and to steer the student from reading towards practice.

Several design decisions in the chapters that follow can be traced directly to this review: strict grounding in uploaded documents with citations (from ChatPDF's example and the RAG literature), a topic model spanning all of a course's documents (the shared gap in Quizlet, ChatPDF and Turbo AI), topic-level mastery driving a 70/30 revision plan (building on Turbo AI's weak-area tracking but at course scope), document-sourced distractors with explanations on every answer (from Kurdi et al.), and past-paper analysis as a first-class feature (absent from every tool reviewed). We also take from this review a sense of proportion. The products we examined are mature and well-engineered, and AGAHI will not out-build them on breadth. Its case rests on being course-aware, verifiable and designed around how students in

Pakistan actually prepare for exams, and the literature suggests that this is a real and unoccupied position rather than a marginal one.

Chapter 4 Software Requirement Specifications

This chapter presents the complete software requirements specification for the AGAHI platform. The requirements documented here were derived from the project objectives, the scope boundaries established in Chapter 2, and the technical decisions made during the early design phase. Each section covers a distinct category of requirements, from the core feature list through to the assumptions that underpin the specification. Whoever reads this chapter should be able to know exactly what the system is required to do and how well it is required to do it and under what conditions.

4.1 List of Features

AGAHI is a document-based study software package. Its capabilities enable a student to go from raw uploaded course material to targeted revision, and to track his/her performance along the way. The following are the main features of the system as delivered in its first release.

- Secure student registration and session-based login mechanism.
- Support for course-related PDFs up to 30 MB or 150 pages per file.
- Automated text extraction, vector embedding generation, and chunk-level document indexing.
- RAG-driven query system restricted to uploaded documents, returning exact page and source citations.
- Automatic creation of structured summaries and interactive flip-style flashcards.
- Multiple-choice question generation utilizing distractors drawn directly from source text.
- Dynamic quiz creation allocating 70% of questions to weak topics and 30% to mastered topics.
- Topic and sub-topic performance tracking displayed on a centralized dashboard with a weakness heatmap.
- Automated examination parser designed to identify recurring high-frequency topics.
- Configurable Pomodoro timer, study streak counter, and immediate post-question feedback.

4.2 Functional Requirements

Functional requirements describe the external operations, inputs, processing logic, and output behaviors of the AGAHI platform. Each requirement specifies the target feature, identifies the primary user executing or triggering the feature, and outlines the mandatory system behavior using formal specification

language.

1. **Student Registration and Authentication**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall deny unauthenticated requests to protected endpoints.
        (b) The system shall allow students to register accounts using university credentials and log in securely.
        (c) The authentication module shall encrypt stored passwords and issue session tokens.
2. **Course Document Ingestion**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall accept text-based PDF uploads, including lecture slides, textbook chapters, course notes, and past papers.
        (b) The upload service shall enforce a strict file size cap of thirty megabytes or one hundred fifty pages per document.
3. **Text Parsing and Vector Embedding Indexing**
    - **Primary User:** System
    - **Description:**
        (a) The system shall extract raw text from ingested PDF files, divide text streams into discrete semantic chunks, and generate dense vector embeddings.
        (b) The vector database shall store and index these embeddings for fast similarity searches.
4. **Context-Restricted Question Answering (RAG)**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall execute Retrieval-Augmented Generation to generate answers strictly bound to the student's uploaded documents.
        (b) The retrieval engine shall filter out irrelevant context and prevent out-of-domain responses.

5. **Source Document and Page Citation**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall append precise document names and page numbers to every generated answer.
        (b) The system shall link each in-text citation to its corresponding source document such that a user click shall trigger the integrated document viewer to open and display the exact cited page segment.
6. **Automated Document Summarization**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall construct structured text summaries from selected PDF documents or specific topics upon student request.
        (b) The summary generator shall highlight key terms and structural hierarchies.
7. **Interactive Flashcard Generation**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall extract core concepts and definitions from uploaded documents to construct digital flashcards.
        (b) The user interface shall display these items as flip cards and log student self-evaluations.
8. **Contextual Quiz Generation**
    - **Primary User:** Student
    - **Description:**
        (a) The system shall build multiple-choice questions from factual document statements.
        (b) The item generation module shall pull distractors directly from surrounding document context rather than generic text.
9. **Adaptive Revision Scheduling**
    - **Primary User:** Student

- **Description:**
  (a) The system shall generate customized revision quizzes using logged student performance history.
  (b) The allocation logic shall assign approximately seventy percent of questions to weak topics and thirty percent to previously mastered topics.

10. **Performance Dashboard and Heatmap**
    - **Primary User:** Student
    - **Description:**
      (a) The system shall record quiz results at topic and sub-topic levels.
      (b) The dashboard shall render a color-coded weakness heatmap and line charts representing historical score trends over time.
11. **Past Paper Frequency Analysis**
    - **Primary User:** Student
    - **Description:**
      (a) The system shall analyze uploaded historical examination papers, categorize questions by subject topic, and calculate frequency scores.
      (b) The interface shall present top-ranked examination topics to guide study focus.
12. **Focus Timer and Engagement Tracking**
    - **Primary User:** Student
    - **Description:**
      (a) The system shall provide a configurable Pomodoro focus timer and track daily study streak counts.
      (b) The quiz module shall deliver immediate visual correctness feedback after every question submission.

4.3 Quality Attributes

The project incorporates four fundamental quality attributes to prevent software defects and guarantee operational efficiency. These metrics evaluate system performance and user experience throughout active deployment.

4.3.1 Reliability

Educational tools require dependable operational stability during intense study sessions. Students rely on AGAHI for exam preparation; system failures cause immediate disruptions. System crashes destroy user focus. Therefore, the backend architecture isolates heavy PDF parsing background workers from the primary user interface. The vector retrieval pipeline processes text asynchronously. This prevents memory spikes when ingesting hundred-page documents. Queries submitted against indexed course files must return contextually relevant answers consistently. Graceful degradation mechanisms handle database timeouts without terminating user sessions. Consequently, the platform maintains steady performance under sustained query loads.

4.3.2 Maintainability

AGAHI employs a modular system architecture to support long-term viability. Machine learning tooling evolves rapidly. Consequently, the core processing engine separates text parsing, vector embedding generation, and language model inference into decoupled modules. Developers can swap vector database providers or integrate updated embedding models without modifying the frontend application. Database schemas for tracking quiz performance utilize normalized relational tables. Clear API boundaries keep code dependencies low. Thus, future developers update individual components without breaking core system workflows.

4.3.3 Usability

Students require a clean workspace free from unnecessary cognitive friction. Visual clutter destroys focus. The user interface minimizes visual distractions to support sustained attention during study sessions. Navigation remains simple. A student uploads a document and initiates revision without configuring complex settings. Performance dashboards translate raw assessment records into intuitive weakness heatmaps and progress charts. Standardized interface components maintain visual consistency across all views. Thus, non-technical users learn system navigation quickly.

4.3.4 Correctness

Academic preparation demands absolute factual accuracy across all generated materials. Hallucinations ruin learning. System responses must derive strictly from student-provided source documents. Therefore, the retrieval pipeline enforces strict contextual bounds that restrict answers to verified document text. Search algorithms reject retrieved passages falling below defined vector similarity thresholds. Every response maps directly to exact document page citations. Quiz algorithms extract multiple-choice

distractors directly from surrounding textbook contexts. The adaptive engine enforces the seventy-thirty question split with mathematical precision.

4.4 Non-Functional Requirements

Non-functional requirements specify qualitative boundaries, performance constraints, and architectural standards for AGAHI. These attributes ensure system stability, security, and responsiveness during actual deployment.

4.4.1 Availability

The system shall maintain an operational availability of ninety-nine percent throughout the academic semester. Exam weeks demand constant uptime. Consequently, the underlying cloud infrastructure balances incoming web traffic across redundant server instances to prevent sudden downtime during peak study hours. Scheduled maintenance windows execute exclusively during off-peak hours between midnight and five morning hours. Automated health checks monitor database connections continuously.

4.4.2 Reusability and Extensibility

The codebase follows a clean modular architecture that separates the user interface, backend service endpoints, vector database interfaces, and language model connectors. Decoupled software components simplify updates. Developers can replace the underlying vector engine or swap language model APIs by altering isolated configuration drivers without touching frontend application code. Utility functions share common parsing logic. This structure allows future engineers to extend platform capabilities with minimal refactoring effort.

4.4.3 Robustness and Fault Tolerance

Data loss ruins student progress. Therefore, the platform implements automatic session recovery to preserve user state and quiz progress if a network connection drops unexpectedly. Corrupted or malformed PDF uploads trigger graceful error handling routines rather than crashing the parsing pipeline. System state persists in real time. Background processing queues retry failed vector embedding tasks three times before logging an error to prevent silent data loss during background operations.

4.4.4 Security and Privacy

Student account authentication relies on JSON Web Tokens and encrypted password storage. Cryptographic hashing protects user credentials. Passwords undergo salted hashing using the bcrypt algorithm

prior to database persistence, preventing plain text exposure during potential data breaches. The system isolates document stores by student identity. Users cannot access external files. Vector database namespaces restrict retrieval-augmented generation queries strictly to documents uploaded by the authenticated user, blocking unauthorized cross-tenant data access completely.

4.4.5 Performance and Scalability

Latency directly impacts study efficiency. The system shall process vector retrieval and return grounded natural language answers in less than three seconds under normal operating conditions. Up to five hundred concurrent active users can query the platform simultaneously without degrading system response times. Background jobs execute asynchronously. PDF parsing pipelines isolate heavy document chunking from the main web thread, keeping page rendering swift while processing large textbook uploads in background worker processes.

4.5 Assumptions

The system specification relies on several operational, technical, and user-level assumptions. These conditions define the operating environment within which AGAHI functions correctly.

1. Uploaded PDF files contain readable, text-based content rather than raw, un-parsed image scans lacking embedded text layers.
2. Students possess basic digital skills, including navigating web browsers, managing local files, and reading graphical dashboard charts.
3. Users maintain an active internet connection capable of supporting standard HTTP requests and file uploads up to thirty megabytes.
4. Students access the platform through modern desktop with JavaScript execution enabled.
5. Uploaded materials directly relate to academic coursework, providing sufficient contextual depth for quiz and summary generation.

4.6 Use Cases

This section describes the major use cases of AGAHI that represent its central study and resource-management functionality. Each use case describes the interaction between the student and the system and focuses on the expected system behavior.

Table 4.1: Use Case (UC-01) – Sign Up

Name	Sign Up		
Actors	Student		
Summary	The student creates an AGAHI account by providing the required registration information.		
Pre-Conditions	The student is not already registered and is not logged in.		
Post-Conditions	A new student account is created and the student can proceed to the login screen.		
Special Requirements	The email address must be valid and the password must satisfy the system's security requirements.		
Basic Flow			
Actor Action		**System Response**	
1	The student selects the "Sign Up" option.	2	The system displays the registration form.
3	The student enters the required name, email address, and password.	4	The system validates the entered information and creates the account.
Alternative Flow			
3	The student enters an email address that is already registered.	4-A	The system displays an error message indicating that the email address is already in use.
3	The student leaves one or more required fields empty.	4-B	The system displays an error message identifying the missing field(s).

Table 4.2: Use Case(UC-02) – Login

Name	Login		
Actors	Student		
Summary	The student accesses AGAHI by providing valid account credentials.		
Pre-Conditions	The student has a registered account and is not currently logged in.		
Post-Conditions	The student is authenticated and is redirected to the AGAHI dashboard.		
Special Requirements	The system shall validate the credentials before creating a user session.		
Basic Flow			
Actor Action		**System Response**	
1	The student selects the "Login" option.	2	The system displays the login form requesting email and password.
3	The student enters the registered email address and password.	4	The system verifies the credentials and redirects the student to the dashboard.
Alternative Flow			
3	The student enters an incorrect email address or password.	4-A	The system displays an error message indicating that the login credentials are incorrect.

Table 4.3: Use Case (UC-03) – Upload Academic Resource

Name	Upload Academic Resource		
Actors	Student		
Summary	The student uploads a text-based PDF containing academic material such as lecture slides, notes, a textbook section, or a past examination paper.		
Pre-Conditions	The student is logged in and the selected file is a PDF within the allowed file size and page limits.		
Post-Conditions	The file is uploaded and added to the student's academic resources for processing.		
Special Requirements	Only text-based PDF files are supported; each file is limited to 30 MB or 150 pages, whichever limit is reached first.		
Basic Flow			
Actor Action		**System Response**	
1	The student selects the "Upload Resource" option.	2	The system displays the upload interface.
3	The student selects and uploads a PDF file.	4	The system validates the file and adds it to the student's resources for processing.
Alternative Flow			
3	The selected file is not a PDF or is an unsupported document type.	4-A	The system displays an error message stating that only supported PDF files can be uploaded.
3	The file exceeds the allowed size or page limit.	4-B	The system informs the student that the file exceeds the permitted 30 MB or 150-page limit.
3	The PDF contains no readable text.	4-C	The system informs the student that image-only or scanned PDFs cannot be processed.

Table 4.4: Use Case (UC-04) – Ask Academic Question

Name	Ask Academic Question		
Actors	Student		
Summary	The student asks a question about the course material available in the student's uploaded resources.		
Pre-Conditions	The student is logged in and at least one relevant academic resource has been processed.		
Post-Conditions	The system displays an answer based on the relevant uploaded material and, where available, identifies the source document and page.		
Special Requirements	The answer generation process shall use the student's uploaded academic material and shall not use external web search results.		
Basic Flow			
Actor Action		**System Response**	
1	The student enters an academic question in the question field.	2	The system accepts the question and prepares it for retrieval.
3	The student submits the question.	4	The system retrieves relevant passages from the uploaded material and displays a response based on the retrieved content.
Alternative Flow			
3	No relevant information is found in the uploaded material.	4-A	The system informs the student that sufficient information could not be found in the uploaded resources.
3	The student submits an empty question.	4-B	The system asks the student to enter a question before continuing.

Table 4.5: Use Case (UC-05) – Generate Study Aids

Name	Generate Study Aids		
Actors	Student		
Summary	The student requests study material such as a summary, flashcards, or practice questions from an uploaded academic resource.		
Pre-Conditions	The student is logged in and the selected academic resource has been successfully processed.		
Post-Conditions	The requested study aid is generated from the selected academic material and is displayed to the student.		
Special Requirements	Generated study aids shall remain based on the student's uploaded academic material.		
Basic Flow			
Actor Action		**System Response**	
1	The student opens a processed academic resource.	2	The system displays the available study-aid options.
3	The student selects a study-aid type and submits the request.	4	The system generates the requested study aid from the selected material and displays it.
Alternative Flow			
3	The selected resource has not finished processing.	4-A	The system informs the student that the resource is still being processed.
3	The student selects an unsupported or unavailable study-aid option.	4-B	The system displays the study-aid options currently available for the selected resource.

Table 4.6: Use Case (UC-06) – Attempt Practice Quiz

Name	Attempt Practice Quiz		
Actors	Student		
Summary	The student takes a practice quiz generated from the academic material and receives feedback on the submitted answers.		
Pre-Conditions	The student is logged in and quiz questions have been generated from processed academic material.		
Post-Conditions	The student's quiz attempt is evaluated and the result and feedback are displayed.		
Special Requirements	The system shall record the student's performance for use in progress and weakness tracking.		
Basic Flow			
Actor Action		**System Response**	
1	The student selects a practice quiz for a course or topic.	2	The system displays the quiz questions and available answer options.
3	The student submits the completed quiz.	4	The system evaluates the answers and displays the score with immediate feedback.
Alternative Flow			
3	The student attempts to submit the quiz without answering a required question.	4-A	The system identifies the unanswered question(s) and asks the student to complete them before submission.
3	The student exits the quiz before submitting it.	4-B	The system does not record the incomplete attempt as a completed quiz result.

Table 4.7: Use Case (UC-07) – Analyze Past Examination Paper

Name	Analyze Past Examination Paper		
Actors	Student		
Summary	The student submits or selects a past examination paper so that AGAHI can identify recurring questions, topics, and examination patterns.		
Pre-Conditions	The student is logged in and a supported past examination paper is available in the student's resources.		
Post-Conditions	The system presents identified recurring questions, topics, and patterns from the available examination papers.		
Special Requirements	The analysis shall be based on past papers available in the student's uploaded resources.		
Basic Flow			
Actor Action		**System Response**	
1	The student selects a past examination paper or a collection of past papers.	2	The system displays the option to analyze the selected papers.
3	The student starts the analysis.	4	The system processes the papers and presents identified recurring questions, important topics, and observable patterns.
Alternative Flow			
3	No suitable past paper is available for analysis.	4-A	The system informs the student that a past examination paper must be uploaded before analysis can be performed.
3	The selected paper cannot be processed.	4-B	The system displays an error message and asks the student to select another supported paper.

Table 4.8: Use Case(UC-08) – View Study Progress

Name	View Study Progress		
Actors	Student		
Summary	The student views academic performance and study progress through the AGAHI dashboard.		
Pre-Conditions	The student is logged in and has recorded study activity or quiz results.		
Post-Conditions	The dashboard displays the student's available progress information, including performance and identified weak areas.		
Special Requirements	Progress information shall be calculated from the student's recorded activity and assessment results.		
Basic Flow			
Actor Action		**System Response**	
1	The student opens the dashboard.	2	The system retrieves the student's available study and performance data.
3	The student selects a course or progress view.	4	The system displays progress indicators, performance information, and weak areas for the selected course.
Alternative Flow			
3	The student has no recorded activity for the selected course.	4-A	The system informs the student that progress information will appear after study or assessment activity is recorded.

Table 4.9: Use Case (UC-09) – Receive Revision Recommendations

Name	Receive Revision Recommendations		
Actors	Student		
Summary	The student views suggested topics for revision based on performance, weaknesses, and patterns found in available past examination papers.		
Pre-Conditions	The student is logged in and sufficient quiz or performance data is available for analysis.		
Post-Conditions	The system presents topics that may require additional revision, together with relevant academic material where available.		
Special Requirements	The 70/30 distribution shall be treated as a guideline for balancing weaker areas with retention of previously learned material.		
Basic Flow			
Actor Action		**System Response**	
1	The student opens the revision recommendations section.	2	The system retrieves the student's performance and relevant past-paper information.
3	The student selects a course for revision guidance.	4	The system presents recommended topics by considering the student's weaknesses and the importance of topics found in available past papers.
Alternative Flow			
3	Insufficient performance data is available.	4-A	The system informs the student that more study or quiz activity is required before personalized recommendations can be generated.

Table 4.10: Use Case (UC-10)– Use Pomodoro Study Timer

Name	Use Pomodoro Study Timer		
Actors	Student		
Summary	The student uses the Pomodoro timer to organize a focused study session.		
Pre-Conditions	The student is logged in or has access to the study dashboard.		
Post-Conditions	The selected timer session completes and the system shows the appropriate session status.		
Special Requirements	The student shall be able to customize the study and break durations.		
Basic Flow			
Actor Action		**System Response**	
1	The student opens the Pomodoro timer.	2	The system displays the timer with the available duration settings.
3	The student selects the desired durations and starts the timer.	4	The system begins the study session and indicates when the study or break interval ends.
Alternative Flow			
3	The student changes the duration before starting the timer.	4-A	The system updates the timer using the selected durations.
3	The student stops the timer before the interval ends.	4-B	The system stops the current interval and returns the timer to its available controls.

4.7 Hardware and Software Requirements

The following sections outline the essential environment configurations needed to deploy and access the application.

4.7.1 Hardware Requirements

The system requires a development or execution machine equipped with a mid-range processor such as an Intel Core i5, along with a minimum of 8 gigabytes of system memory and 10 gigabytes of local storage space. A stable internet connection is necessary to handle external application programming interface calls to machine learning engines and remote data layers. End users can access the application from any standard personal computer or laptop with basic hardware capable of handling interactive web graphics.

4.7.2 Software Requirements

The development and runtime environments rely on an integrated development tool like Visual Studio Code and Git for source code control under a modern operating system like Windows. The web application requires Node.js to bundle and serve the React user interface, while the application backend runs on Python or higher using the FastAPI framework. Database storage relies on a PostgreSQL database equipped with the pgvector extension and the Supabase client library, while the system requires an updated web browser such as Google Chrome on the client side to properly view the application.

4.8 Graphical User Interface

GUI sample of the system is as under:

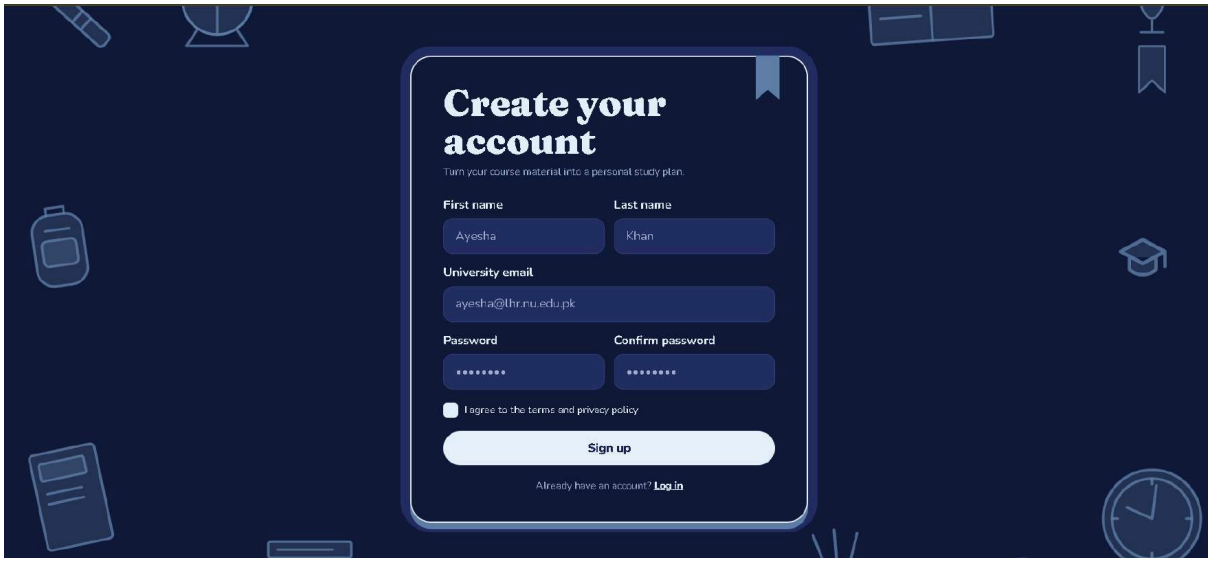


Figure 4.1: Signup Page
This illustration covers UC-01

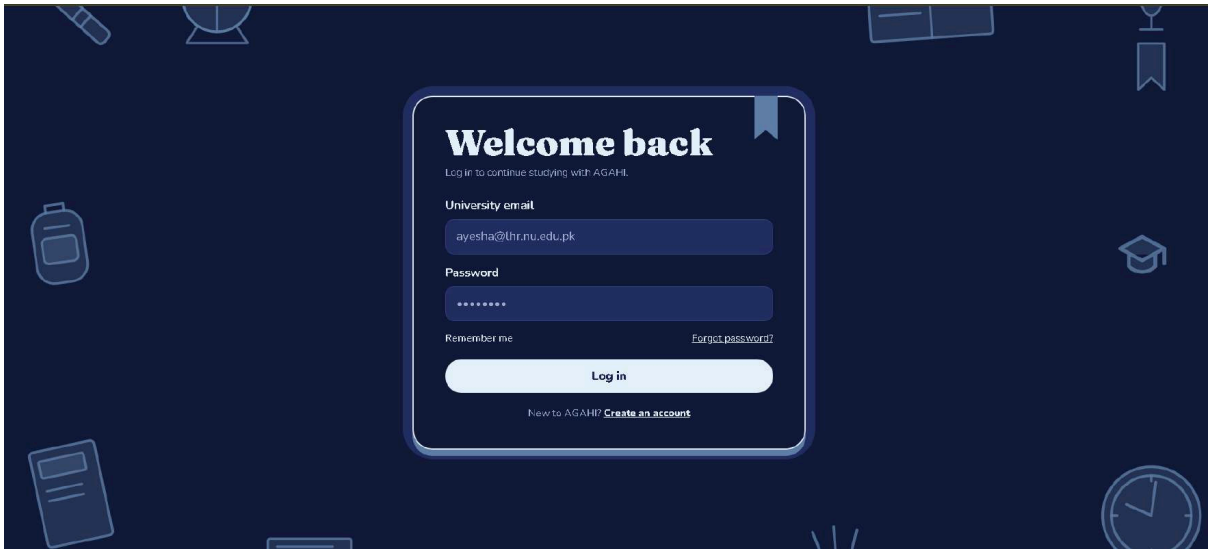


Figure 4.2: Login Page
This illustration covers UC-02

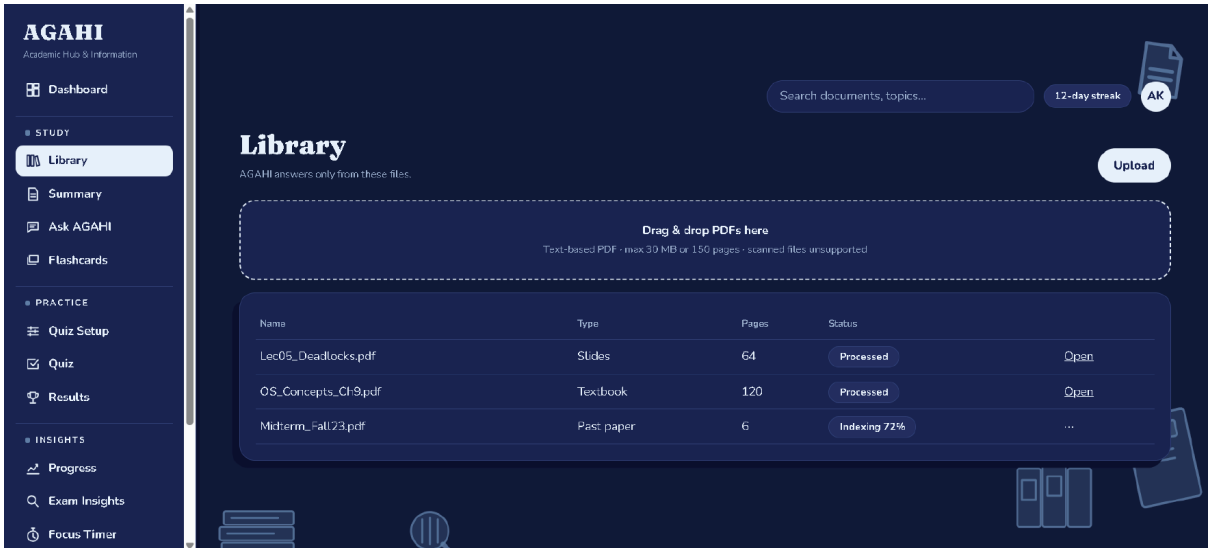


Figure 4.3: Library Page
This illustration covers UC-03

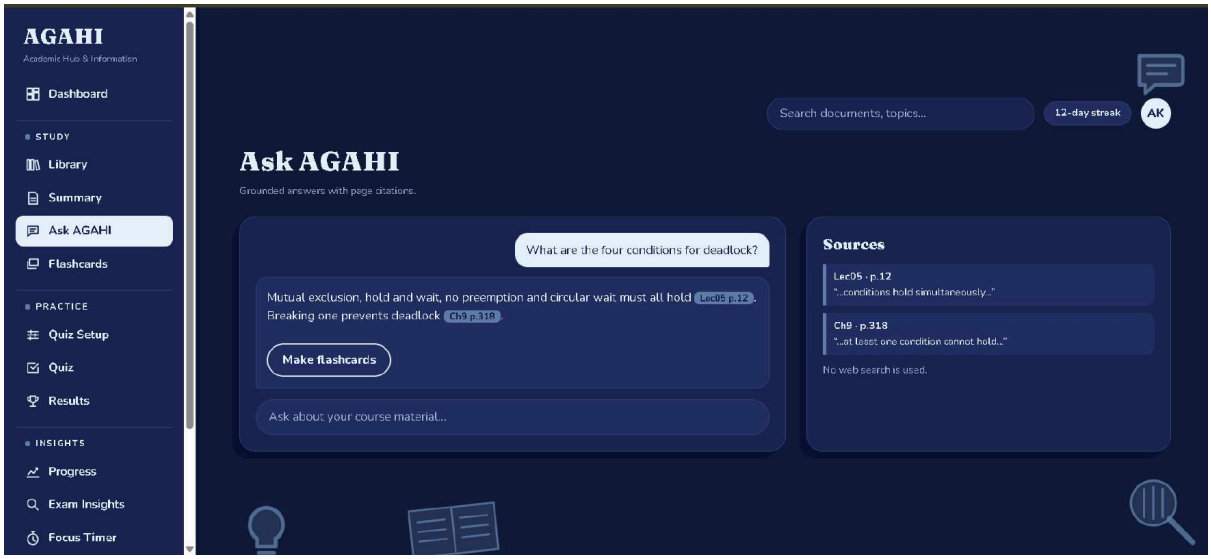


Figure 4.4: Ask AGAHI Page
This illustration covers UC-04

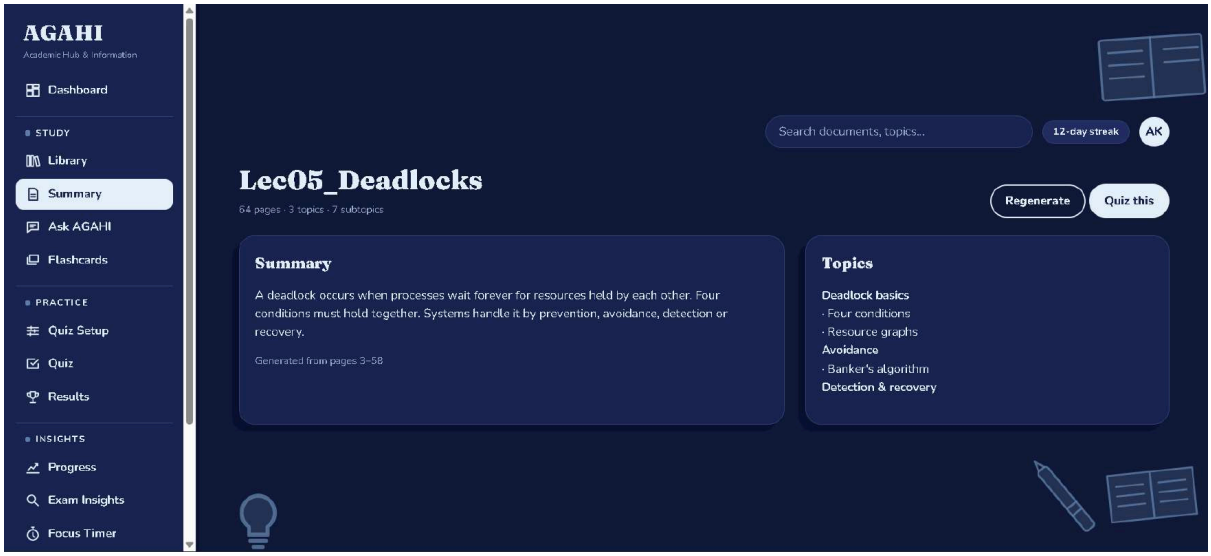


Figure 4.5: Summary Page
This illustration covers UC-05

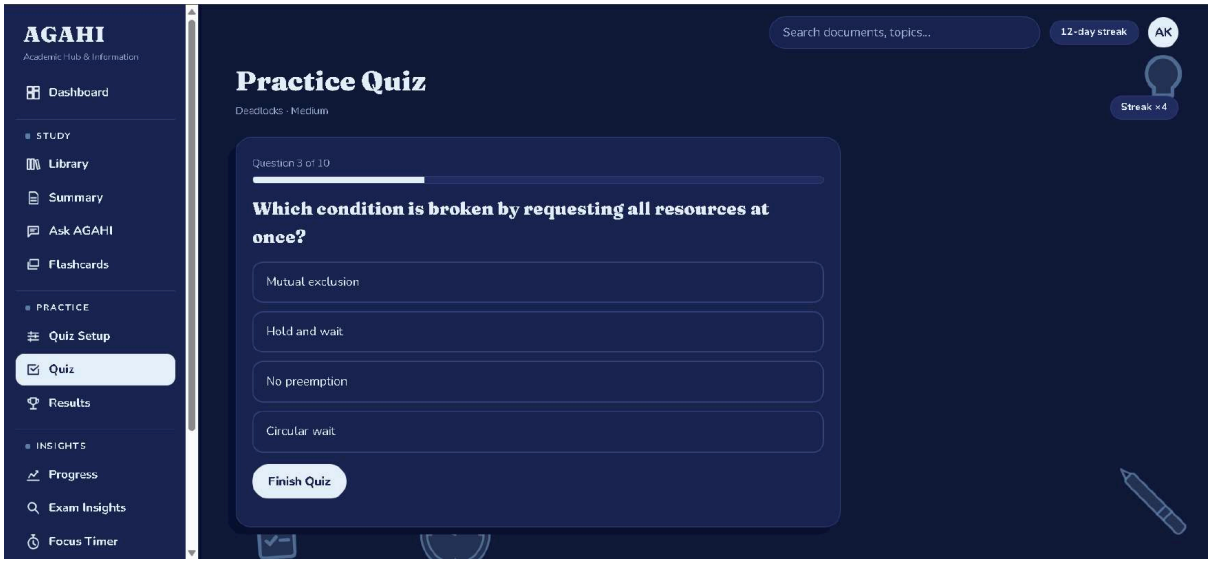


Figure 4.6: Quiz Page
This illustration covers UC-06

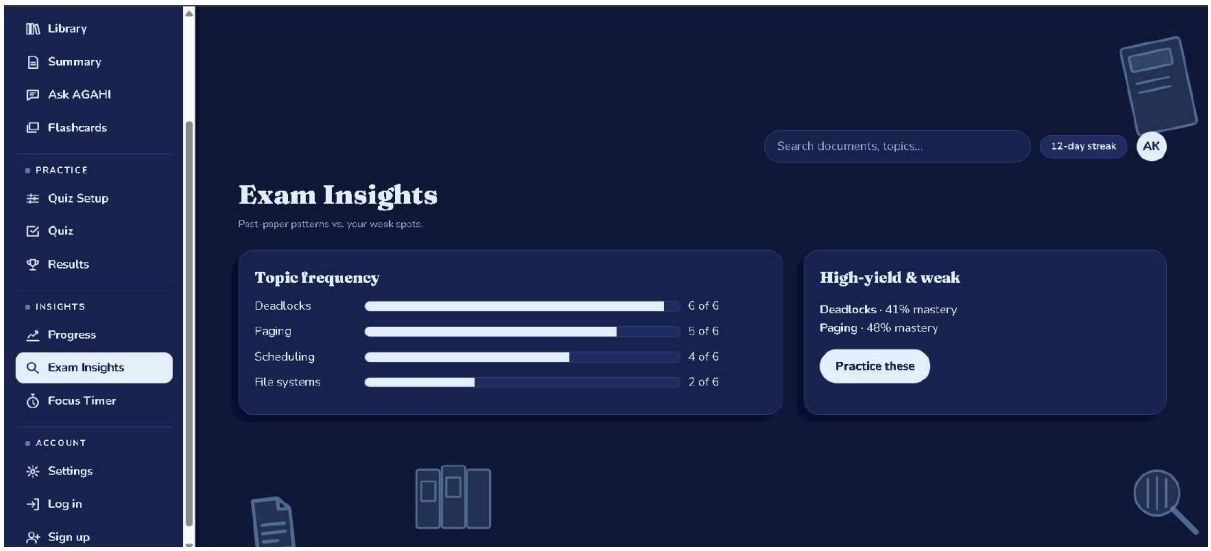


Figure 4.7: Exam Insights Page
This illustration covers UC-07

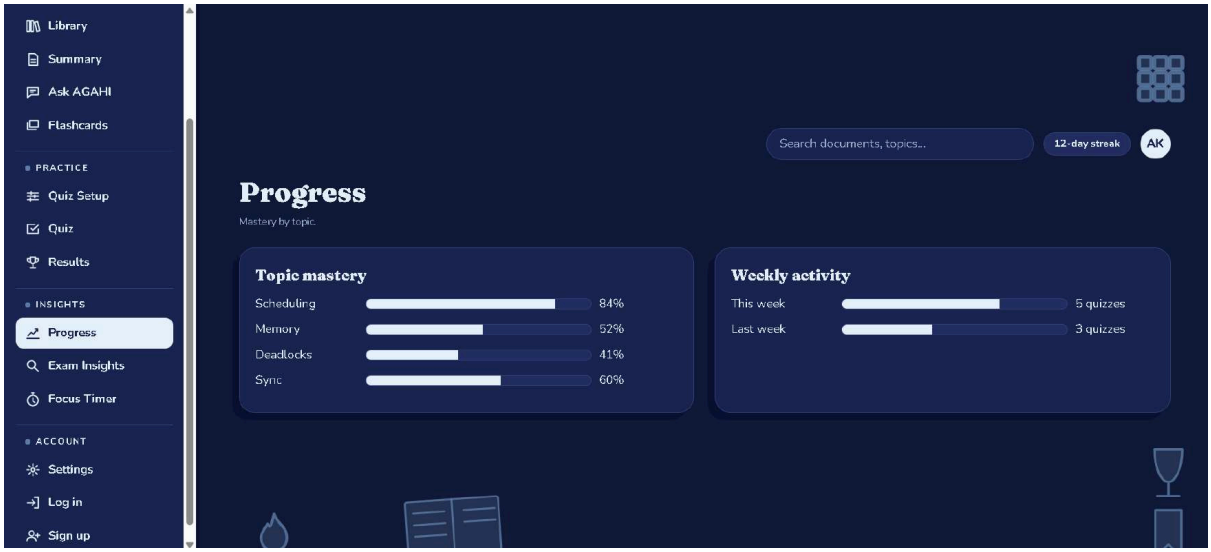


Figure 4.8: Progress Page
This illustration covers UC-08

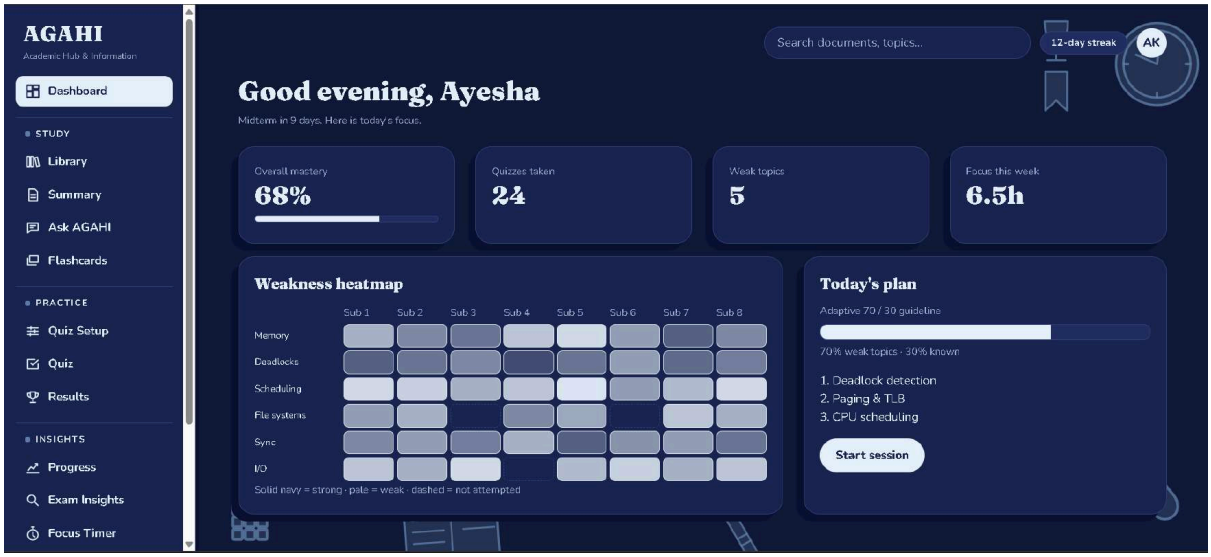


Figure 4.9: Dashboard Page
This illustration covers UC-09

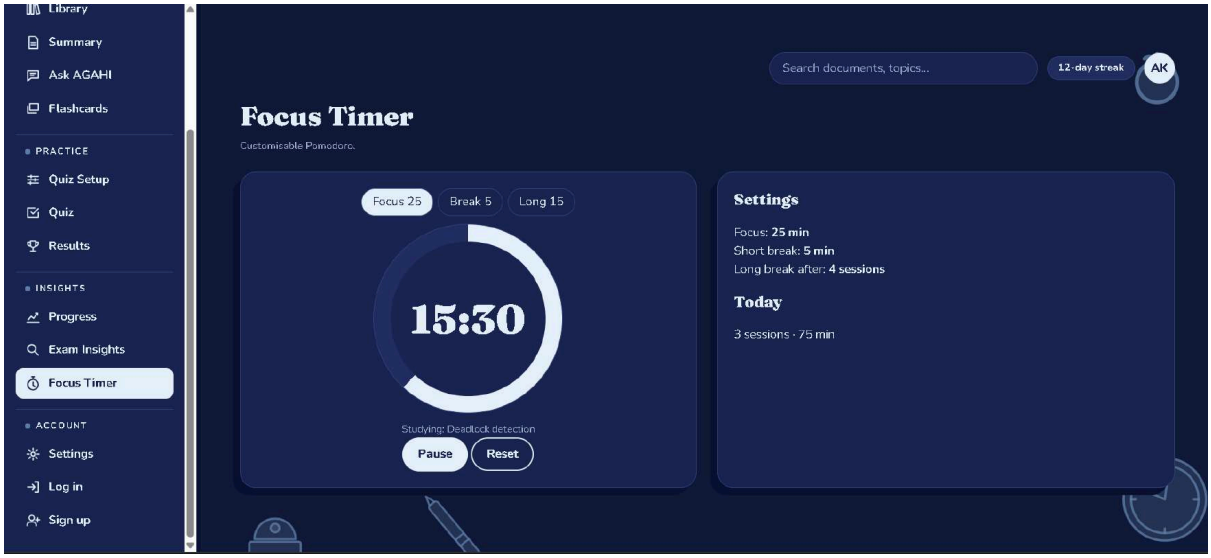


Figure 4.10: Focus Timer Page
This illustration covers UC-10

4.9 Database Design

4.9.1 ER Diagram

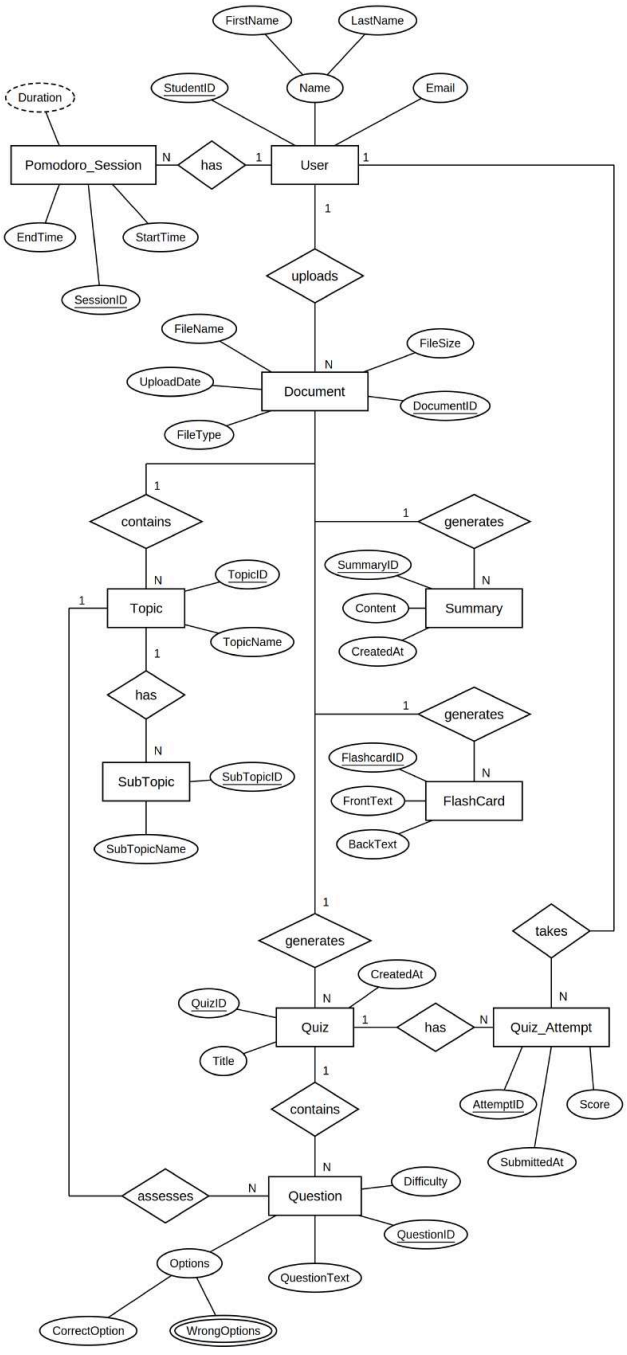


Figure 4.11: ER Diagram
This figure shows the ER Diagram

4.9.2 Data Dictionary

This table contains all the entities along with their attributes in the below table:

Table 4.11: Data Dictionary Table

The given table gives the details of the tables of the database

Entity	Attributes	Data Type	Relation To	Nullable	Description
User	StudentID	Integer		No	Id of the student (PK)
	FirstName	String		No	First name of the student
	LastName	String		No	Last name of the student
	Email	String		No	Email of the student (unique)
Pomodoro_Session	SessionID	Integer		No	Id of the session (PK)
	StudentID	Integer	User	No	Id of the student (FK)
	StartTime	DateTime		No	Start date and time
	EndTime	DateTime		Yes	End date and time
	Duration	Integer		N/A	Derived: EndTime minus StartTime
Document	DocumentID	Integer		No	Id of the document (PK)
	StudentID	Integer	User	No	Id of the uploader (FK)
	FileName	String		No	Name of the uploaded PDF
	FileType	String		No	Type of the file (PDF)
	FileSize	Integer		No	File size in MB (max 30 MB)
	UploadDate	Date		No	Date of upload
Topic	TopicID	Integer		No	Id of the topic (PK)
	DocumentID	Integer	Document	No	Id of the source document (FK)
	TopicName	String		No	Name of the topic
SubTopic	SubTopicID	Integer		No	Id of the sub-topic (PK)
	TopicID	Integer	Topic	No	Id of the parent topic (FK)
	SubTopicName	String		No	Name of the sub-topic
Summary	SummaryID	Integer		No	Id of the summary (PK)
	DocumentID	Integer	Document	No	Id of the source document (FK)
	Content	String		No	Text of the summary
	CreatedAt	DateTime		No	Date and time of creation
FlashCard	FlashcardID	Integer		No	Id of the flashcard (PK)
	DocumentID	Integer	Document	No	Id of the source document (FK)
	FrontText	String		No	Text on the front of the card
	BackText	String		No	Text on the back of the card
Quiz	QuizID	Integer		No	Id of the quiz (PK)
	DocumentID	Integer	Document	No	Id of the source document (FK)
	Title	String		No	Title of the quiz
	CreatedAt	DateTime		No	Date and time of creation
Question	QuestionID	Integer		No	Id of the question (PK)
	QuizID	Integer	Quiz	No	Id of the quiz (FK)
	TopicID	Integer	Topic	No	Id of the assessed topic (FK)
	QuestionText	String		No	Text of the question
	Difficulty	String		No	Difficulty level of the question
	Options	Composite		No	Composite attribute made up of CorrectOption and WrongOptions
	CorrectOption	String		No	Component of Options: the correct answer option
	WrongOptions	String (multi-valued)		No	Component of Options: multivalued attribute holding the wrong answer options
Quiz_Attempt	AttemptID	Integer		No	Id of the attempt (PK)
	QuizID	Integer	Quiz	No	Id of the quiz (FK)
	StudentID	Integer	User	No	Id of the student (FK)
	Score	Float		No	Score of the attempt
	SubmittedAt	DateTime		No	Date and time of submission

4.10 Risk Analysis

AGAHI seeks to help students organize and prepare academic materials such as notes and revision quizzes, and monitor learning progress. One way to improve reliability and resilience of this system is to identify the risks that might affect its operation and development.

4.10.1 Technical Risks

The most important technical issues include those which are linked to PDF processing, information extraction, and AI-generated content. The system must be able to extract data from academic PDF files uploaded by users and retrieve the relevant data for providing summarized information (flashcards and quizzes). Any malfunction in the process of information extraction might lead to omitting significant information. Also, the process of extraction may not provide sufficient context for generating the right answer. It is also important to mention that the content generated by AI may be incomplete or incorrect. To avoid the risk of these occurrences, AGAHI will allow processing only plain text PDFs that will not be larger than the pre-defined limit. Additionally, the system will produce content based on the provided materials and try to include page numbers and references to the original source.

4.10.2 Risks Associated with Data and Privacy

AGAHI will keep academic materials uploaded by students as well as information connected to their quizzes and learning achievements. These materials can be made up of information regarding a student's notes, works, lecture materials, and previous tests. Unpermitted access to the above documents may jeopardize the students' privacy and their academic achievements. Thus, there is a need to apply suitable authentication and access regulations that will allow users to use only their own documents. Moreover, the process of usage and storing uploaded documents and progress data must be managed throughout the development of the system.

4.10.3 Risks of Development and Project

AGAHI consists of several closely interlinked components: document processing, retrieval based on RAG technology, creation of studying materials, quizzes, adaptive learning, analysis of previous exams, and progress dashboard. Therefore, to include and join the components within the running project time frame will be a complicated task. Problems in one module will influence the other modules as well. In order to avoid such problems, we start with developing core features of the system: from uploading the PDF document and using it for generation of content, testing quizzes, to keeping track of individual progress.

Chapter 5 High-Level and Low-Level Design

This chapter presents the design of AGAHI, starting from a general view of what the system does and how it is organised, moving through the considerations and constraints that shaped the design, and then on to the architecture, the domain model and the detailed interaction design. The intention is that a reader who has finished the requirements specification in the previous chapter should be able to see how each requirement maps on to a part of the system, and that a developer joining the project should be able to understand where a given responsibility lives before reading any code.

5.1 System Overview

AGAHI operates as an online learning platform that transforms existing course materials into structured study artifacts. Students upload text-based files, including lecture slides, instructor notes, textbook chapters, and past examination papers. The system processes these raw documents into an organized, searchable representation. Consequently, AGAHI generates contextual summaries, answers to factual inquiries, flashcards, and diagnostic quiz questions directly from user-supplied texts. The platform also aggregates student performance across successive assessments. It builds a detailed profile of student strengths and weaknesses, tracks topics that recur across past examination papers, and recommends targeted study topics. A minimalist user interface minimizes distraction through bite-sized evaluation modules and an integrated timer based on the Pomodoro technique. The platform functions across six core subsystems that correspond to the use cases established in the preceding chapter.

5.1.1 Document Ingestion and Extraction of Topics

Document ingestion establishes the operational foundation of the entire system. Ingestion begins with validation. When a student uploads a file, validation filters verify that the document conforms to a valid PDF structure, contains an extractable text layer, and stays within thresholds of 30 MB or 150 pages, whichever boundary the file crosses first. The ingestion service immediately rejects scanned files or image-only documents, providing explicit feedback to prevent unextractable content from entering downstream pipelines. Once validated, text extraction proceeds page by page. The parser organizes text into topics and subtopics, divides the content into overlapping chunks, and attaches source page numbers and topic identifiers to each passage. These segmented chunks undergo vectorization and persist within the vector store alongside their positional metadata. Because parsing multi-page documents demands processing time, asynchronous background workers manage the ingestion queue while presenting real-time progress indicators to the user.

5.1.2 Grounded Question Answering

Students submit questions regarding one or more processed documents. The pipeline converts each query into an embedding vector and retrieves the most semantically similar text chunks from the vector store. The language model synthesizes an answer using only those retrieved passages. System prompts strictly instruct the model to declare when the provided text lacks the required information. The interface displays the generated answer alongside the exact document titles and page numbers from which the evidence originated, enabling students to verify every factual claim directly against their source material. This implementation instantiates the Retrieval-Augmented Generation (RAG) pattern examined in Section 3.1, isolating queries to a per-user corpus and preventing the generator from falling back on parametric memory.

5.1.3 Study Aid Generation

Students can request structured summaries or active recall flashcards from any indexed document. The language model synthesizes both artifacts directly from the extracted text and underlying topic hierarchy. The database caches these generated study aids immediately, allowing users to review them later without incurring redundant model inference costs. Flashcards exist as discrete question-and-answer pairs tied to the source document and indexed by page numbers wherever available.

5.1.4 Adaptive Practice and Mastery Tracking

Quizzes serve as the diagnostic mechanism through which AGAHI profiles student understanding. Prior to question generation, the analytics service computes a mastery metric for each topic based on historical quiz performance. It partitions curriculum topics into weak and mastered categories, assigning approximately 70% of test items to deficient topics and 30% to mastered ones. Questions follow a multiple-choice format generated from source text chunks. Distractors derive from related concepts within the same course documents, ensuring options remain plausible rather than arbitrary. Each generated question includes an explanatory rationale and a citation to its source page. Students receive immediate feedback upon selecting an option. Once a quiz completes, the system logs the final result and recalibrates the mastery ratings for all tested concepts. This 70/30 allocation balances targeted remediation with retention maintenance.

5.1.5 Past-Paper Analysis and Revision Guidance

Students upload past examination papers through the standard ingestion pipeline, with the option to trigger dedicated comparative analysis. The analysis engine isolates individual exam questions, clusters

them against established course topics, and calculates both topic frequency and question recurrence across historical semesters. This process constructs an empirical overview of examination tendencies. The user dashboard unifies this historical data with the student's mastery profile. A visual weakness heatmap displays performance levels across every topic. Concurrently, the recommendation algorithm prioritizes revision topics by balancing individual performance deficits against the historical weight of each topic in past assessments.

5.1.6 Focus and Engagement Features

The user interface emphasizes brief, structured study interactions. Bite-sized quiz modules, visual progress indicators, streak counters, and instantaneous feedback sustain engagement. A configurable Pomodoro timer allows students to define study and break intervals while recording completed focus blocks directly into their performance logs. These lightweight features accommodate students who struggle with prolonged study sessions, including learners with ADHD. Their uncluttered presentation avoids introducing interface distractions.

5.1.7 Basic Design Approach

Structurally, AGAHI is a three-tier web application. The presentation tier is a single-page React application running in the student's browser; it holds no credentials for external services and never talks to the language model directly. The application tier is a Python backend built with FastAPI. It consists of a set of controllers, one per functional area (authentication, documents, question answering, study aids, quizzes, exam analysis, dashboard and Pomodoro), implemented as thin route handlers, and the services they delegate to: a parsing service for ingestion, a RAG service for retrieval and grounded generation, a recommendation engine, and narrow adapters around the vector store, the embedding model and the language model. Controllers orchestrate; domain entities such as `Document`, `Topic`, `Quiz`, `Question`, `QuizAttempt` and `PomodoroSession` hold state and the behaviour that belongs to that state, and nothing more. The data tier is a hosted Supabase project, which provides authentication, a PostgreSQL database for users, documents, topics, quizzes, attempts and sessions, file storage for the uploaded PDFs, and, through the pgvector extension of the same database, the vector store for chunk embeddings. Embeddings are produced by the all-MiniLM-L6-v2 sentence-embedding model served through the Hugging Face Serverless Inference API, and language-model inference is obtained from the Groq API; both are consumed as external services. This separation was chosen so that the expensive and volatile parts of the system, the model providers, sit behind narrow interfaces and can be replaced without touching the rest of the design, and so that the per-user isolation of data, which matters for both privacy and copyright, is enforced at the data tier rather than left to application code. The remainder

of this chapter describes this organisation in detail, and the reasoning behind the individual technology choices is given under Architectural Strategies.

5.2 Design Considerations

Before settling on a design we had to be clear about what we were assuming, what we could not change, and what we were optimising for. This section records those decisions. Several of them follow directly from the constraints listed in Section 2.7 and from the literature reviewed in Section 3.2; where that is the case we say so rather than repeating the argument.

5.2.1 Assumptions and Dependencies

The design rests on the following assumptions and depends on the following external components. If any of them turns out to be false, the part of the design that relies on it will have to be revisited.

- **Related software and hardware.** AGAHI is delivered through a current web browser on a laptop or desktop computer; no software is installed on the student's machine. The client is a React application and the server is a Python application built on FastAPI. Beyond these, the system depends on four external components: a hosted Supabase project, which provides authentication, the PostgreSQL database, file storage and, via pgvector, the vector store; the Hugging Face Serverless Inference API, which serves the all-MiniLM-L6-v2 model used to embed document chunks and queries; the Groq API, which serves the large language model used for answering, summarising and question generation; and a Python PDF text-extraction library used during ingestion. We do not train, fine-tune or host any models ourselves. The design assumes that these services remain available with their current programmatic interfaces and that their free tiers, on which the project currently runs, continue to permit the volume of calls a small user base generates.
- **Operating systems.** Because the client is a web application, no particular operating system is assumed on the student's side beyond the ability to run a modern browser with JavaScript enabled. The backend is ordinary Python and runs on any platform that FastAPI supports; the team does not administer an operating system of its own.
- **End-user characteristics.** Users are university students in Pakistan who are comfortable with web applications and have used general-purpose AI chatbots, but who should not need to understand retrieval or language models to use the system. Course material is assumed to be in English and in text-based PDF form. Study habits vary widely, from steady revision through the semester to intensive preparation in the days before an exam, and some users have difficulty sustaining attention for long periods. The design therefore assumes short sessions, immediate feedback and

a forgiving interface rather than long, uninterrupted use.

- **Course material.** We assume that a course's material, taken together, covers what the course examines. AGAHI answers and generates only from uploaded documents, so if a student uploads an incomplete set of slides the system will correctly report that information is missing rather than invent it. We also assume that the uploaded material is something the student is entitled to use for personal study, and the design treats it accordingly: stored per user, served only to that user, and never shared or used for training.
- **Possible and probable changes.** The most likely change is to the model providers. Pricing, rate limits and model quality in this area change quickly, and it is probable that the language model served through Groq, or the embedding model, will be swapped at least once during the project. The design isolates each provider behind an adapter for this reason, so that such a swap is a change to one configuration file and one adapter module. Other probable changes are adjustments to the file-size and page limits as we learn how the infrastructure behaves under load, and refinements to the 70/30 revision ratio, which the proposal already describes as a guideline rather than a fixed rule. Changes that are possible but out of scope for the present project include support for scanned or non-PDF material, Urdu-language processing, a mobile client and integration with institutional learning management systems; the design does not preclude them, but it does not make provision for them either.

5.2.2 General Constraints

The constraints below have a direct effect on the shape of the design. Each is stated together with the design consequence that follows from it.

- **Hardware and software environment.** The team has no GPU hardware and no budget to run its own models, so all embedding and language-model inference happens through external APIs. Consequently the application tier is designed to be light: it orchestrates calls and stores results, and the computationally heavy work is pushed either to the provider or to pgvector's similarity search inside the database. Because most request time is spent waiting on these external calls, the backend uses FastAPI's asynchronous request handling so that one slow provider call does not block other students' requests.
- **End-user environment.** Students will use AGAHI on campus and at home over connections of varying quality. Uploads of up to 30 MB are therefore treated as background jobs with visible progress and status, every long-running operation returns control to the user immediately, and the client avoids transferring large payloads repeatedly; generated summaries and flashcards are

stored once and fetched, not regenerated on each visit.

- **Availability and volatility of resources.** The Groq and Hugging Face free tiers impose rate limits and the Supabase free tier has quotas of its own, and any move beyond those tiers would carry a cost per token or per request. The design caps the number of chunks passed to the model in a single prompt, bounds the length of generated output, caches generated artefacts, and keeps the per-document limits of 30 MB and 150 pages so that the cost and time of ingesting a single file are predictable. Should a provider be unavailable, the affected feature fails with a clear message while the rest of the system (viewing documents, existing summaries and flashcards, the dashboard, the timer) continues to work, which is also how the availability and fault-tolerance requirements of the previous chapter are met.
- **Standards compliance.** The only input standard the system depends on is the PDF format with a text layer. Internally, the design is documented in UML, following the conventions used in the department's software design courses, and the database is described by the entity-relationship model and data dictionary in the previous chapter. Backend code follows PEP 8 and the frontend follows the React conventions set out under Policies and Tactics. On the policy side, the system is designed to respect academic-integrity expectations: it reinforces the course as taught and does not help a student bypass it.
- **Interoperability and interface requirements.** No integration with learning management systems or other institutional software is required or planned. The React client communicates with the FastAPI backend over HTTPS using JSON; the backend communicates with Supabase through its client library and with the Groq and Hugging Face services over their published HTTPS APIs. All provider credentials live on the server side only.
- **Data repository and distribution.** All persistent data lives in the hosted Supabase project: PostgreSQL tables for users, documents, topics, sub-topics, summaries, flashcards, quizzes, questions, attempts and Pomodoro sessions, as laid out in the data dictionary; object storage for the original PDF files; and a pgvector-indexed table holding each chunk's embedding together with its text, document, topic and page number. Keeping relational data and embeddings in one database avoids a second service to operate and lets a retrieval query be filtered by owner and document in the same statement. Every row and file is owned by exactly one student and access rules at the database layer enforce that a student can only read their own data. There is no data sharing between accounts in the present scope.
- **Security and privacy.** Students authenticate through Supabase's authentication service; sessions are carried by JSON Web Tokens, passwords are stored only as salted bcrypt hashes, as the re-

quirements specify, and the client never holds provider API keys. Uploaded material may be copyrighted by instructors or publishers, so it is stored privately, is used only to serve the uploading student, and is not used to train or improve any model. Account deletion must remove the student's documents, embeddings and derived artefacts.

- **Memory and capacity limitations.** Two model limits shape the pipeline. The all-MiniLM-L6-v2 embedding model accepts an input of at most 256 word-piece tokens and truncates anything longer, which sets the upper bound on chunk size and is why documents are split into overlapping chunks of a few sentences rather than whole pages. The language model's context window bounds how much retrieved text can accompany a question, which is why only the top-ranked chunks are supplied. The file-size and page limits bound the number of chunks a single document produces. All of these are configuration values rather than constants scattered through the code, so that they can be tuned as we learn more about real course material.

- **Performance requirements.** The requirements set a target of under three seconds for a grounded answer under normal load and of up to five hundred concurrent users. The design meets the first by making a question cost exactly one embedding call, one indexed similarity search and one language-model call, with nothing else on the critical path, and the second by keeping the backend stateless and asynchronous so that it spends its time waiting rather than computing. Ingestion and past-paper analysis are allowed to take longer and are therefore designed as asynchronous background jobs with status reporting. Quiz generation sits in between: it is triggered explicitly by the student, who is shown progress while questions are produced.

- **Network communications.** All communication is client-to-backend and backend-to-provider over HTTPS. The browser never calls a model provider directly, both to protect credentials and so that grounding instructions and citation handling cannot be bypassed from the client.

- **Verification and validation.** Because the central promise of the system is that its outputs are grounded, testing must check that promise, not only that the code runs. The design therefore makes every generated answer carry the chunks it was based on, so that an automated check can confirm that cited pages exist and that the answer text is supported by them. The testing strategy described under Policies and Tactics follows from this: unit tests for the deterministic parts (chunk splitting, the 70/30 allocation), integration tests of the full retrieval-and-generation cycle against a fixed document with known content, and black-box tests of the user-facing flows. Generated quiz questions will additionally be assessed against a stated protocol (relevance, correctness, clarity, distractor plausibility) following the recommendations of the question-generation literature reviewed in Section 3.2 [9].

- **Other quality goals.** The requirements call for a tool that students will actually keep using. The design addresses this through the short-session interaction model, immediate feedback on every quiz answer, and progress that is visible on the dashboard, rather than through features that would add complexity.

5.2.3 Goals and Guidelines

A small number of principles governed the design and were used to settle disagreements when they arose. We state each with the reason behind it.

- **Grounding before everything else.** Every answer, summary, flashcard and question must come from the student's documents and be traceable to a page. This is the property that makes AGAHI usable for exam preparation, where a confident wrong answer is worse than no answer, and it is the main lesson of the RAG literature reviewed in Section 3.2 [3, 5]. When a design choice would have made the system more capable but less verifiable, verifiability won.
- **Course-aware, not just document-aware.** Existing tools operate on a single file, set or note. AGAHI's data model is organised around topics that span all of a course's documents, so that mastery, past-paper analysis and recommendations can be expressed at the level a student actually revises at. This is the gap identified in the literature review and it is the reason the `Topic` entity sits at the centre of the domain model.
- **Practice over reading.** The learning-science evidence reviewed in the previous chapters favours self-testing and spaced review over re-reading. The design therefore makes it easy and quick to go from a document to a quiz or flashcards, and the dashboard and recommendations always point towards something to practise rather than something to read again.
- **Keep it simple.** The team is three students working over two semesters alongside coursework. We preferred one well-made path through each feature to several configurable ones, and we chose a hosted backend and pgvector inside the existing database over self-managed infrastructure and a separate vector service, so that effort goes into the study features rather than operations. Simplicity also serves the users: students revising the night before an exam do not want to learn a tool.
- **Responsiveness over cost over memory.** Where speed and cost conflict, interactive operations favour speed and background operations favour cost. Memory use is not a significant concern in a hosted web application of this size and was not optimised for.
- **Familiar where possible.** Chat-style question answering, flashcards that flip and multiple-choice quizzes are patterns students already know from the tools reviewed in the literature review. The

design reuses these interaction patterns deliberately, with all major features reachable from one persistent sidebar, so that what is new about AGAHI, the grounding and the course-level adaptivity, is the only thing a student has to get used to.

- **Privacy by default.** Uploaded material is personal and often copyrighted. Per-user isolation is enforced at the data layer, nothing is shared between accounts, and nothing uploaded is used to train models. These are design rules, not optional settings.
- **Replaceable providers.** Because the model landscape changes quickly, the language model (currently via Groq), the embedding model (currently all-MiniLM-L6-v2 via Hugging Face) and the vector store (currently pgvector) are each accessed through a narrow interface of our own so that any of them can be swapped with a local change.

5.2.4 Development Methods

The project proceeds across two distinct phases aligned with the standard academic semesters of the Final Year Project curriculum. The first phase, documented in this report, concentrates on requirements analysis, architectural synthesis, and system specification. During this stage, the developers elicited use cases, formulated the core domain model, designed relational database schemas, and modeled dynamic system interactions using sequence diagrams. Concurrently, the team established the technology stack, configured the initial Supabase environment, and implemented an early prototype of the PDF ingestion pipeline. This prototype resolved a major technical uncertainty early. The team needed empirical proof that native extraction and chunking routines could parse heterogeneous academic slide decks effectively before proceeding with downstream components. The second phase implements the remaining platform components against this specification and executes empirical evaluation.

System architecture follows conventional object-oriented analysis and design methodologies. Analysis began with structured use cases. From these specifications, the developers derived conceptual domain entities and entity-relationship diagrams, using system sequence diagrams to delineate clear boundaries between user actions and platform entry points. Design-level sequence diagrams subsequently distributed concrete responsibilities across controllers, background services, and domain entities. Responsibility assignment follows established modular design principles. Controllers operate strictly as coordinators. Domain entities maintain internal state variables, whereas external network operations reside exclusively within specialized service classes. An earlier structural iteration placed vector retrieval logic directly inside the `Document` entity. Code reviews quickly exposed this design as an emerging god-object antipattern; consequently, the architecture redistributed those operations into dedicated parsing and RAG service modules. Structural models follow notation standards taught within departmental

software architecture coursework.

For the retrieval and generation pipeline, the architecture incorporates the foundational Retrieval-Augmented Generation paradigm introduced by Lewis et al. [3], incorporating practical implementation recommendations documented by Gao et al. [5]. The pipeline applies dense passage retrieval over segmented text chunks, enforces a bounded top-k contextual window, and applies context-only prompting routines. For dense vectorization, the team selected the all-MiniLM-L6-v2 model from the Sentence-BERT family [13]. This model provides adequate semantic clustering while operating efficiently within the free-tier inference constraints of serverless infrastructure. The developers evaluated and discarded domain-specific fine-tuning. Fine-tuning demands substantial labeled academic datasets alongside computational resources that exceed project resources; in addition, prompt engineering and citation verification enforce grounding constraints far more reliably than weight modification. Quiz generation mechanisms adopt principles from the systematic review by Kurdi et al. [9], which guides both distractor generation and qualitative evaluation metrics for generated assessment items.

Day-to-day engineering follows an iterative and incremental software lifecycle. Short development cycles conclude with functional milestone demonstrations presented during weekly supervisor review sessions. These review sessions evaluate deliverables and recalibrate immediate technical objectives. Source code resides on GitHub and follows a structured Git feature-branch workflow. Developers review pull requests prior to merging changes into the primary production branch. Similarly, the team maintains sequence diagrams as text-based PlantUML source files, ensuring diagrams remain versioned alongside corresponding backend implementations. A rigid plan-driven waterfall methodology was rejected early. Generative model outputs across idiosyncratic course materials exhibit inherent unpredictability; therefore, an inflexible development model cannot accommodate the empirical adjustments that language model integration continuously demands.

5.3 System Architecture

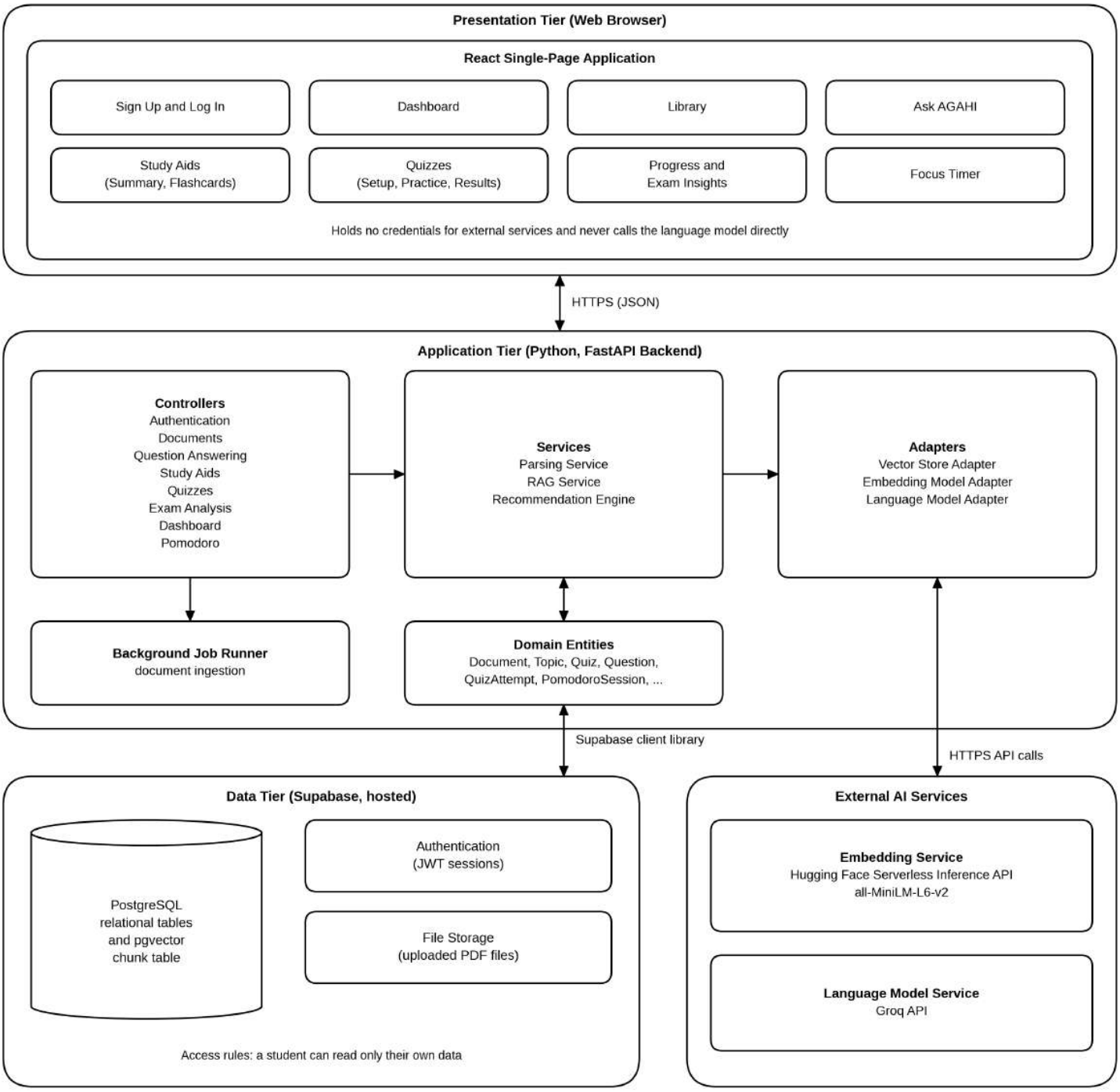


Figure 5.1: Architecture Diagram
System's high level diagram

The system follows a three-tier architecture structured around a Presentation Tier, an Application Tier, and a Data Tier, alongside external AI dependencies. The user interacts directly with a React single-page application that handles the user interface, study aids, and quizzes, ensuring no direct communication with or exposed keys to external language models. This presentation layer sends requests via HTTPS to a Python FastAPI backend application tier, which utilizes controllers to route requests, active services to manage parsing and recommendation engines, and background job runners to process heavy document ingestion tasks. The application tier interacts securely with the data tier through a Supabase client library to access a PostgreSQL database containing relational tables, text vectors, and user file storage, where row-level access rules restrict students to reading only their own data. Finally, the system leverages structural adapters within the application layer to securely make external API calls out to Hugging Face serverless inference for text embeddings and the Groq API for language model processing.

5.3.1 Subsystem Architecture

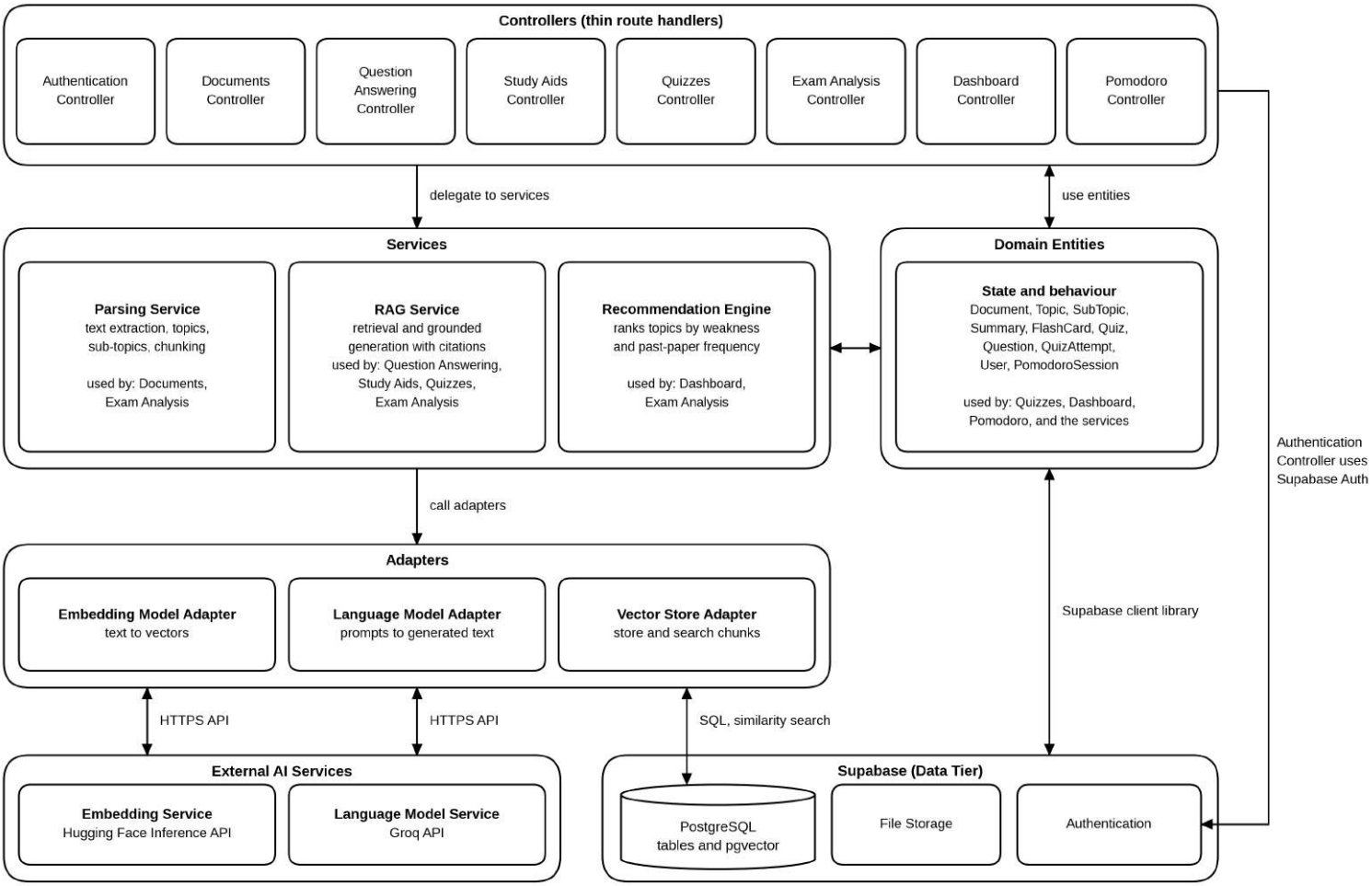


Figure 5.2: Application Tier Subsystem
Detailed component interaction showing controllers, services, and structural adapters

The application backend is built using a decoupled service-oriented architecture where thin route controllers delegate business logic directly to domain services while interacting with system entities. Incoming HTTP requests hit specific route handlers inside the Controllers layer, such as the Authentication, Documents, or Question Answering controllers, which rely on defined Domain Entities to maintain structural state and tracking metrics like user sessions, quiz attempts, and document topics. These controllers offload specific functional execution to the Services layer, where the Parsing Service extracts structural text and chunks documents, the RAG Service handles vector-grounded retrieval and citation generation, and the Recommendation Engine computes weaknesses based on past exam patterns. To decouple internal logic from specialized external platforms, these services route operations through an

Adapters layer containing dedicated modules for text-to-vector embedding mapping, language model prompt formatting, and vector store chunk querying. Finally, these adapters translate calls into standard protocols, interacting with external AI microservices via HTTPS APIs for Hugging Face embedding generation and Groq language generation, while simultaneously interacting with the Supabase Data Tier using a unified client library and direct SQL similarity searches to modify underlying database tables, object storage files, and authentication tokens.

5.4 Architectural Strategies

This section outlines the key design decisions made for AGAHI. Each strategy directly influences the overall structure of the system.

5.4.1 Programming Language and Framework Selection

The backend is built in Python using FastAPI. Python was chosen because the core AI operations, including embedding generation, vector search, and RAG pipeline construction, rely on libraries that are Python-native. FastAPI handles asynchronous requests efficiently, which matters when the system is waiting on external API responses. The frontend is built in React due to its component-based structure, which suits a dashboard-heavy interface with multiple independent widgets.

5.4.2 Database Strategy

AGAHI uses Supabase, which provides managed PostgreSQL with the pgvector extension. This single database handles both relational data, such as user accounts and quiz records, and vector embeddings for semantic search. A separate vector database was considered but rejected because pgvector covers the same need without adding another service to maintain.

5.4.3 AI Pipeline Architecture

The system follows the Retrieval-Augmented Generation pattern. Embeddings are generated using all-MiniLM-L6-v2 through the Hugging Face Serverless Inference API. Language model inference runs through the Groq API. Both were selected because they provide sufficient capability within free-tier constraints.

5.4.4 Concurrency Strategy

FastAPI's native async support allows the backend to handle multiple requests concurrently. Since several operations involve waiting on external APIs, async handling prevents one slow call from blocking

others. This keeps the system responsive under normal load without requiring additional infrastructure.

5.4.5 User Interface Paradigm

The interface is designed so that all major features are accessible from a persistent sidebar. A student can move between the document uploader, quiz runner, flashcard viewer, and dashboard without leaving the platform. The weakness heatmap uses colour intensity to represent performance directly, so the student does not need to interpret a legend.

5.4.6 Version Control

GitHub is used for hosting the project repository. Git is used for version control with a feature-branch workflow, where changes are reviewed before merging into the main branch. This keeps the codebase stable and the team synchronized throughout development.

5.5 Domain Model/Class Diagram

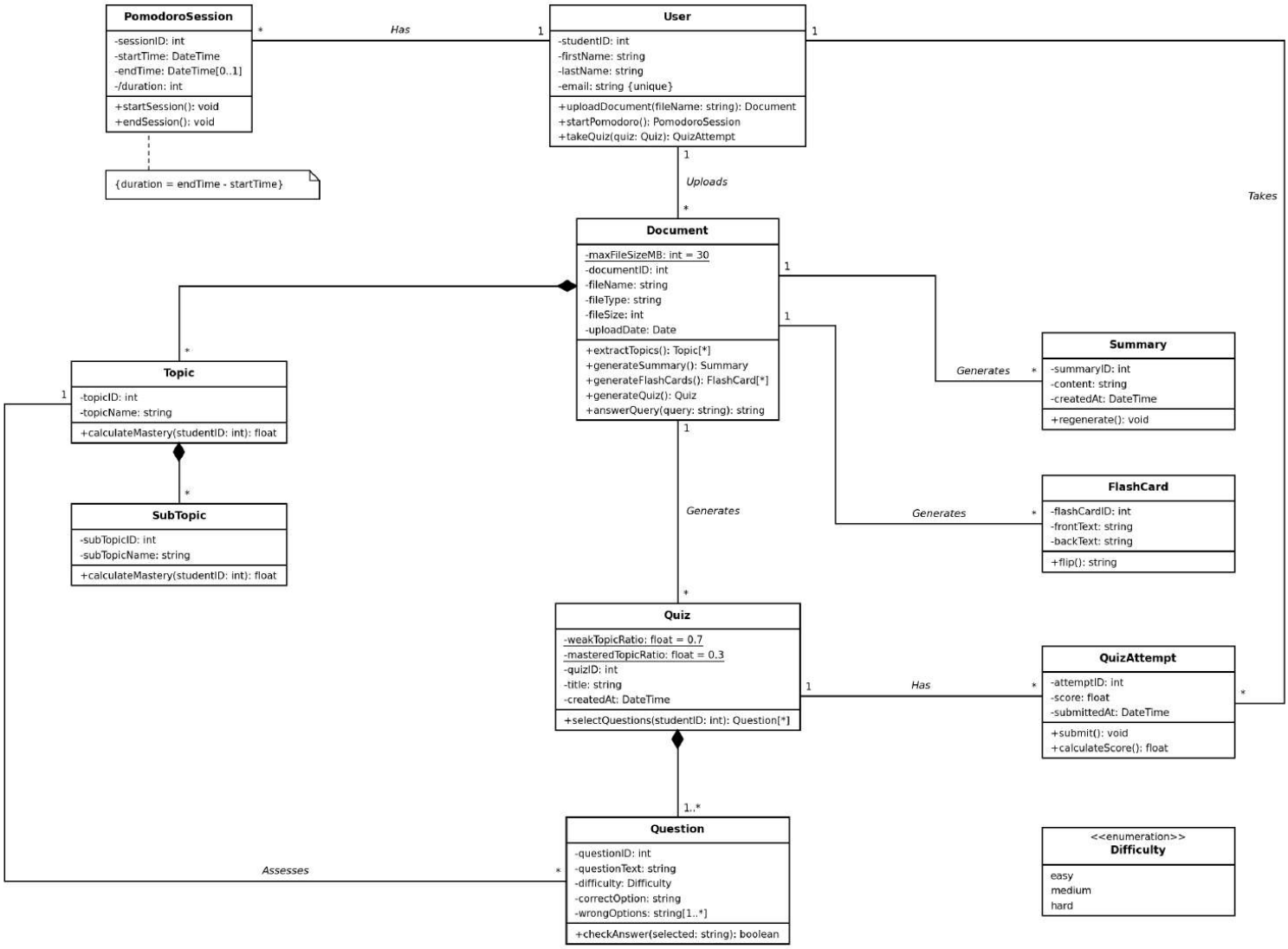


Figure 5.3: Class Diagram
The class diagram of system is given above

5.6 Sequence Diagrams

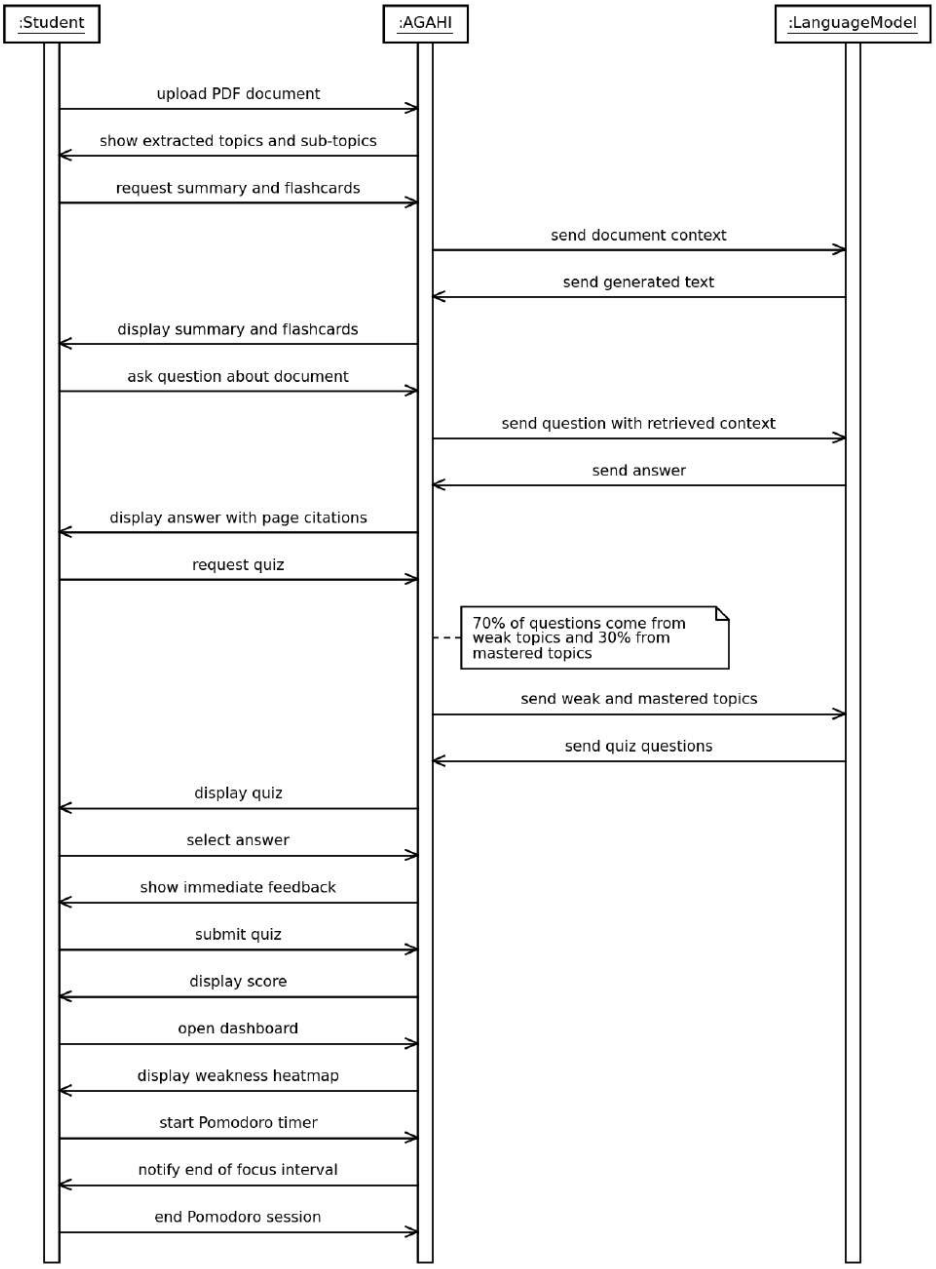


Figure 5.4: System Level Sequence Diagram
The sequence diagram of the system is given above

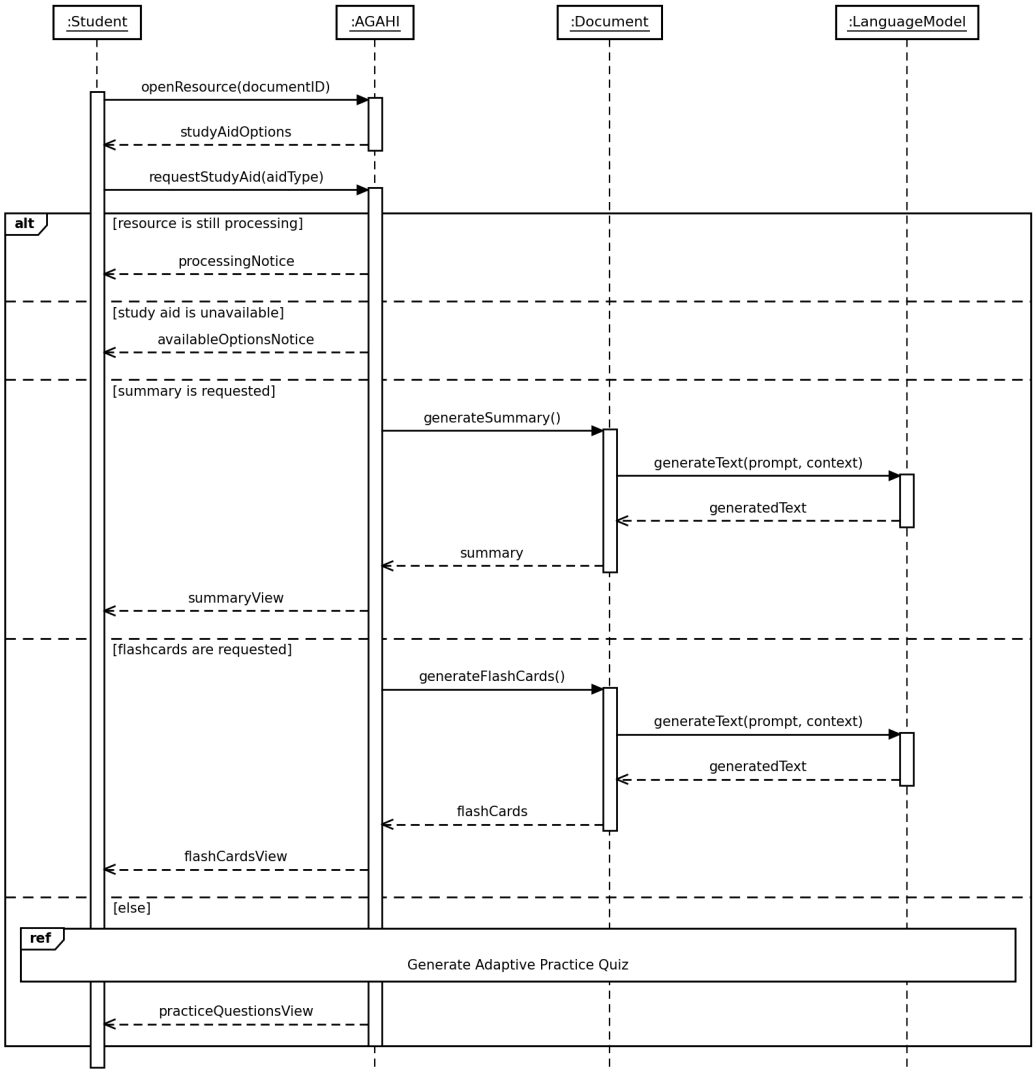


Figure 5.5: Generate Study Aids

The sequence diagram of the generation of the study aids is given above

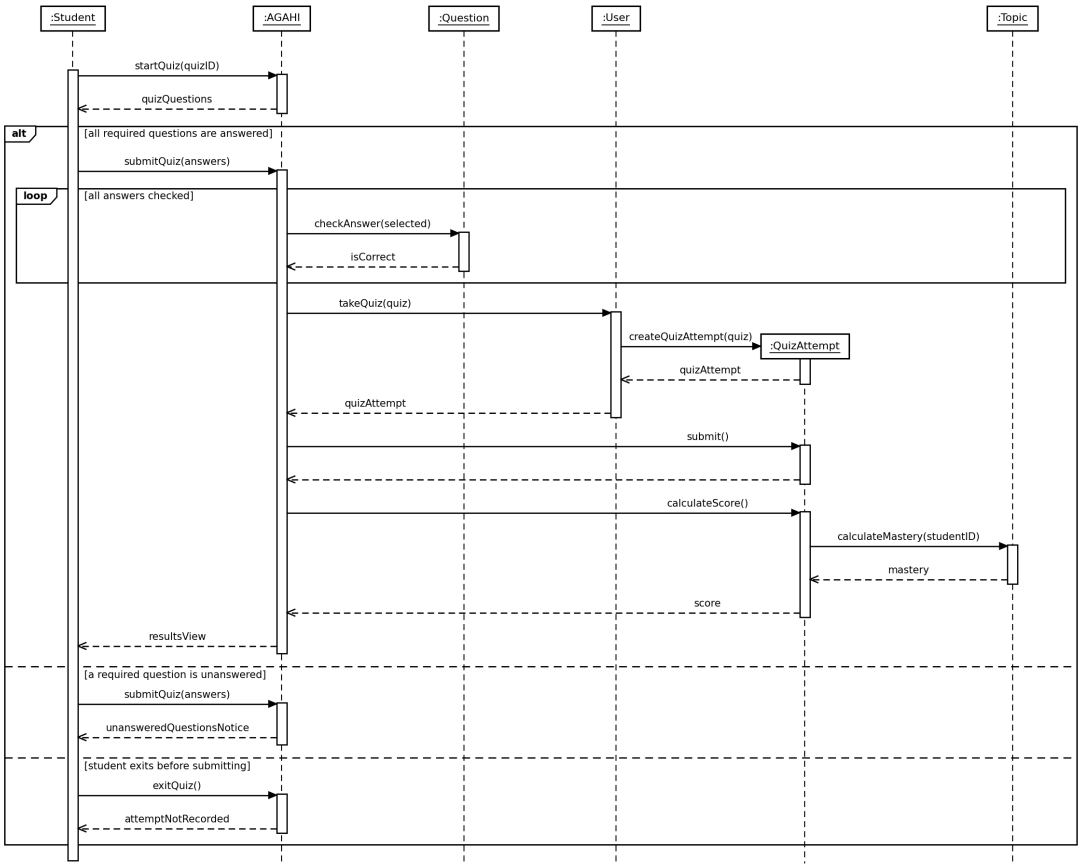


Figure 5.6: Attempt Practice Quiz

The sequence diagram of attempting of the practice quiz is given above

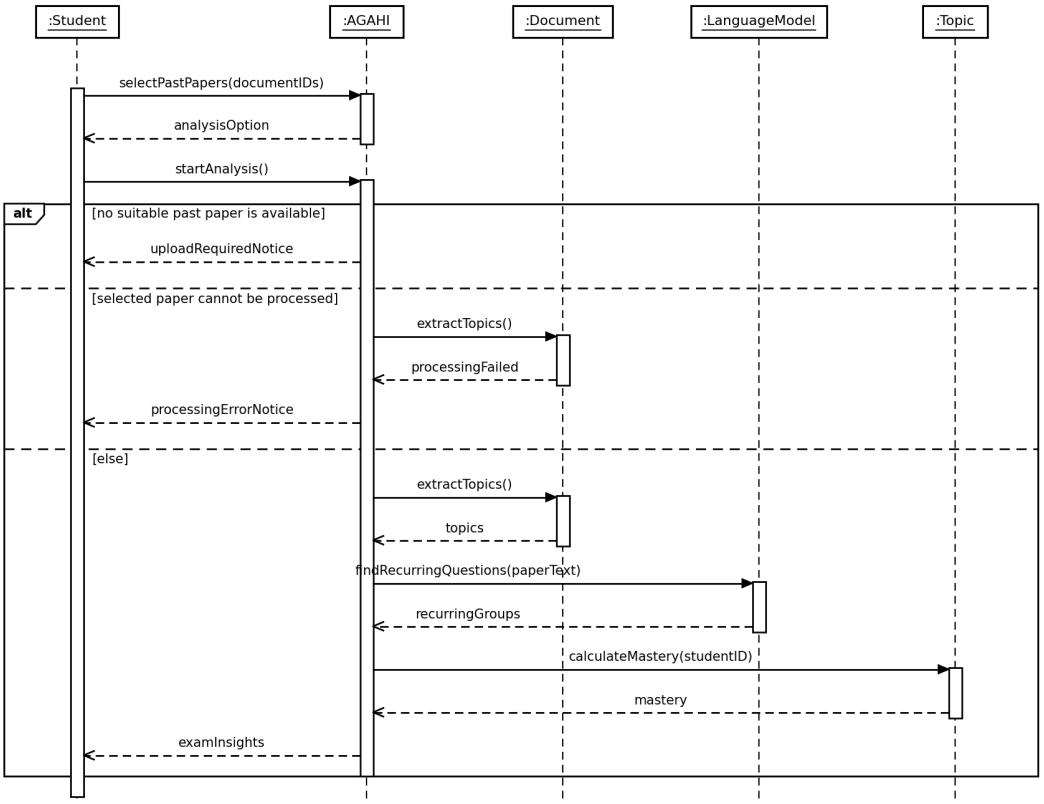


Figure 5.7: Analyze Past Examination Paper

The sequence diagram of the analysis of the past examination paper is given above

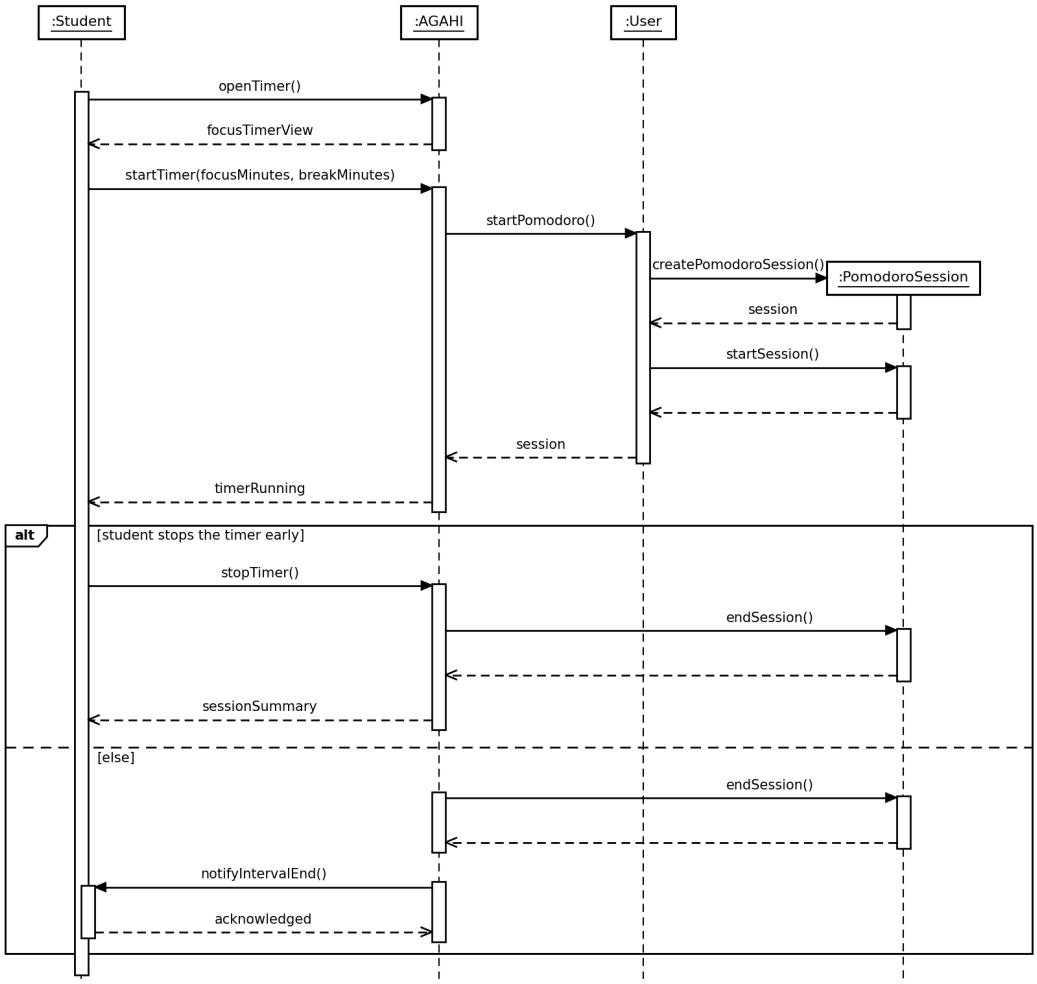


Figure 5.8: Use Pomodoro Study Timer
The sequence diagram of using of the Pomodoro study timer is given above

5.7 Policies and Tactics

This section covers implementation-level decisions that shape how components are built and maintained without altering the overall architecture.

5.7.1 Coding Environment

Visual Studio Code is the development environment for both the frontend and backend. It handles Python and JavaScript in a single editor without requiring a paid license. ESLint and Prettier are used on the frontend for consistent formatting, and Pylance is used on the backend for type checking.

5.7.2 Coding Guidelines and Conventions

Backend code follows the PEP 8 style guide. Route handlers are kept thin, with business logic extracted into separate service modules. Frontend components follow a functional pattern with hooks. Shared logic such as API wrappers and authentication state is managed through custom hooks rather than duplicated across components.

5.7.3 Error Detection and Recovery

FastAPI exception handlers catch known error types and return structured responses with readable messages. A global handler catches unhandled exceptions and logs them without exposing internal details to the client. On the frontend, API failures surface as inline messages so the student receives specific feedback rather than a blank screen.

5.7.4 Testing Strategy

Unit tests cover deterministic functions such as chunk splitting and the adaptive revision allocation algorithm. Integration tests cover the full RAG cycle using a fixed test document with known content. Black-box tests verify the complete user-facing flows, including document upload, quiz completion, and dashboard update.

5.7.5 Extensibility

The ingestion pipeline is structured so that adding a new file format requires only a new extraction adapter. The quiz engine separates generation from rendering, so a new question format can be added without modifying the adaptive revision logic. Both decisions keep future changes isolated to single modules.

5.7.6 Software Maintenance

Dependency versions are pinned in the project requirements files so that updates are deliberate. Swapping the embedding model or language model requires changes in one configuration file and one adapter module, not edits spread across the codebase. This keeps the system maintainable as the underlying AI services evolve.

Bibliography

[1] Quizlet Inc., "Quizlet: Learning tools and flashcards." Available: https://quizlet.com [Accessed September 30, 2026].

[2] B. Waluyo and S. Bucol, "The impact of gamified vocabulary learning using Quizlet on low-proficiency learners," *Computer Assisted Language Learning Electronic Journal*, vol. 22, no. 1, pp. 164–185, 2021.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey." arXiv preprint arXiv:2312.10997, 2023.

[6] ChatPDF, "ChatPDF: Chat with any PDF." Available: https://www.chatpdf.com [Accessed September 30, 2026].

[7] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.

[8] Turbo AI, "Turbo AI (formerly Turbolearn AI): AI note taker and study tools." Available: https://www.turbo.ai [Accessed September 30, 2026].

[9] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020.

[10] Anki, "Anki: Powerful, intelligent flashcards." Available: https://apps.ankiweb.net [Accessed September 30, 2026].

[11] Google, "Gemini Notebook (formerly NotebookLM)." Available: https://notebooklm.google [Accessed September 30, 2026].

[12] Notion Labs, Inc., "Notion AI." Available: https://www.notion.com/product/ai [Accessed September 30, 2026].

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.

[14] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, "Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1–17, 2023.

[15] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[16] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.

[17] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, "Distributed practice in verbal recall tasks: A review and quantitative synthesis," *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.

[18] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology," *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.